

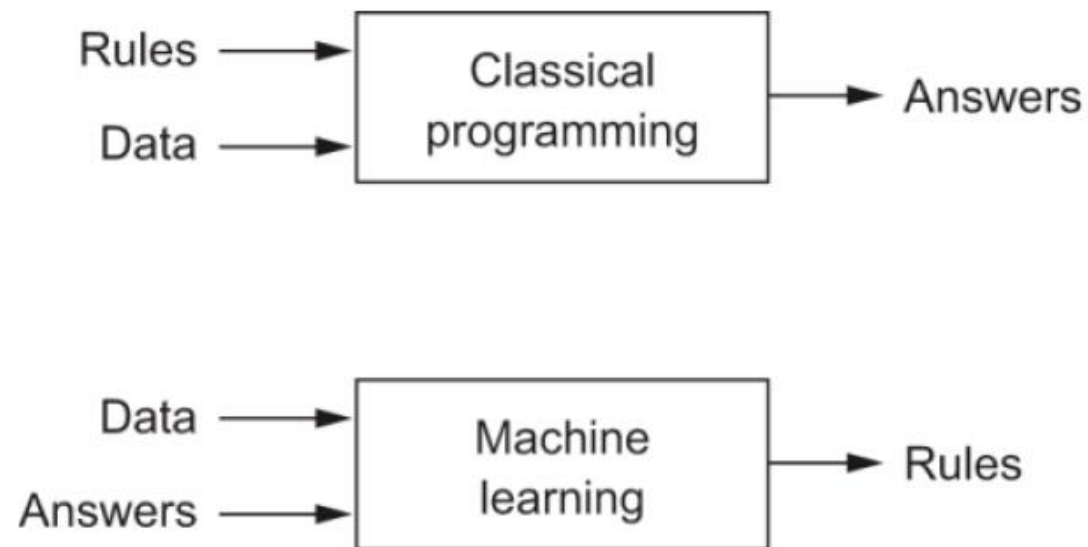
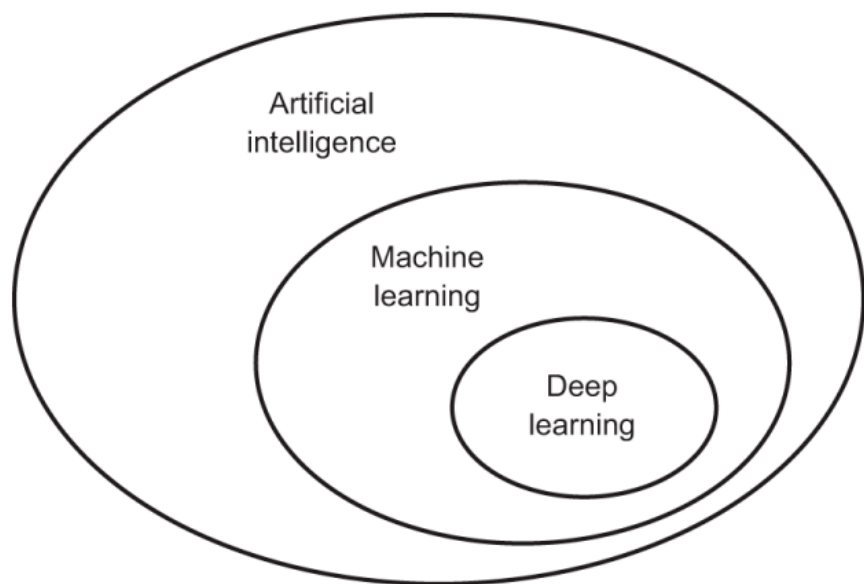
>>>>

CLASE 9.

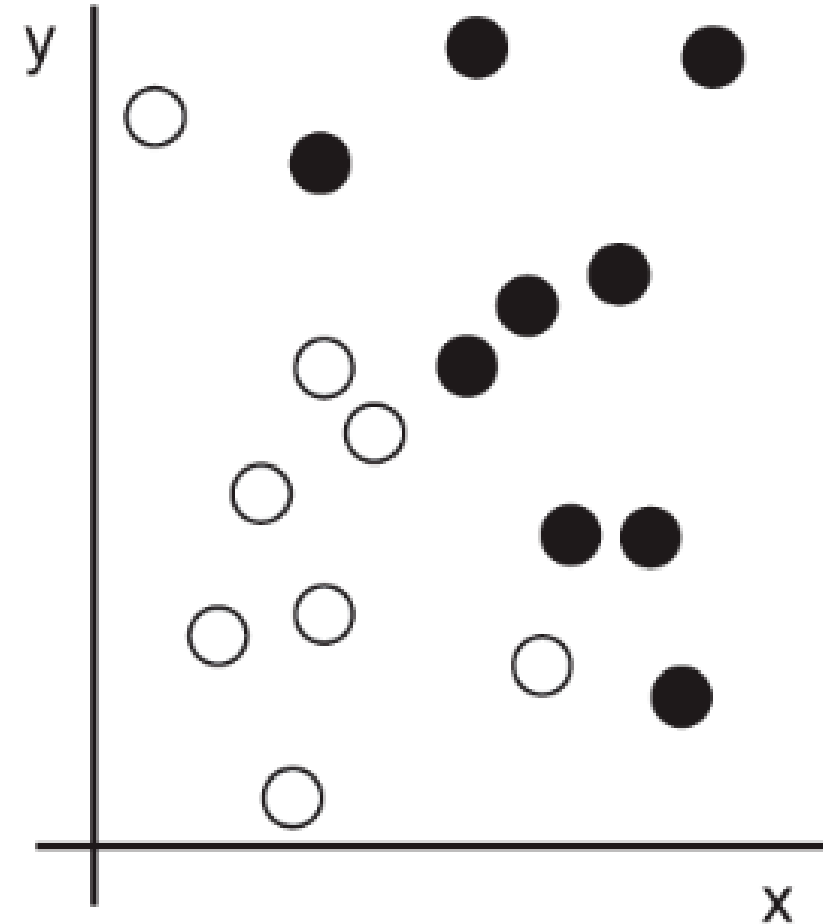
REDES NEURONALES – APRENDIZAJE PROFUNDO

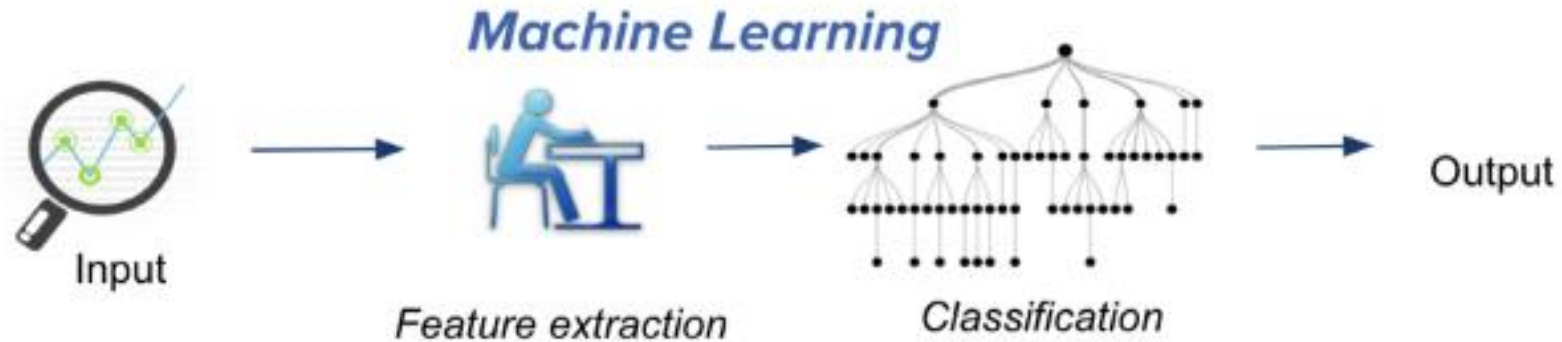


ARTIFICIAL INTELLIGENCE, MACHINE LEARNING, AND DEEP LEARNING

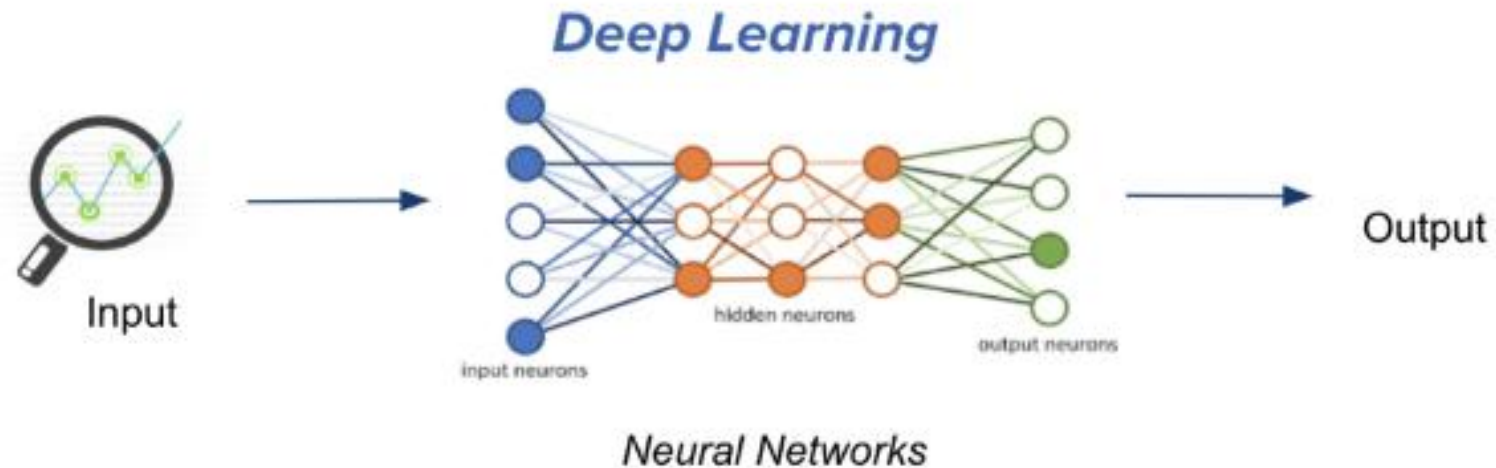


- 1.Dataset de entrada.
- 2.Ejemplos de la salida
- 3.Representación de características.**
- 4.Una manera de medir si el algoritmo esta realizando un buen trabajo o no.

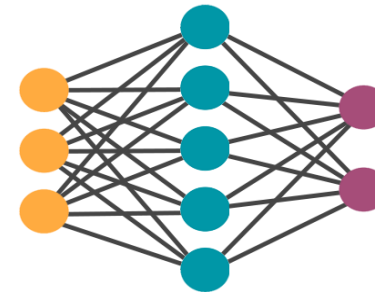
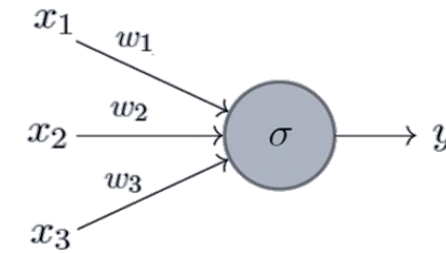
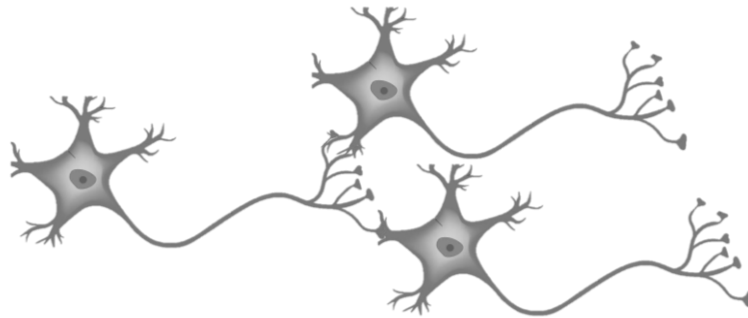
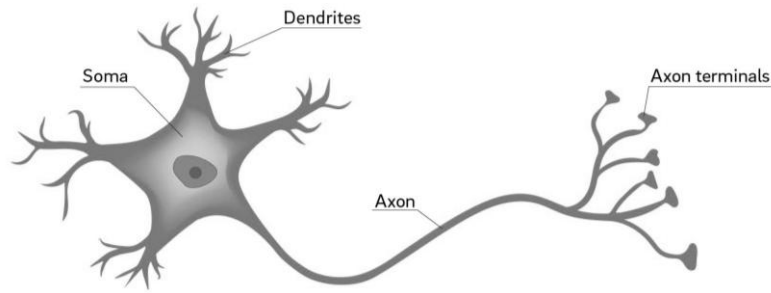


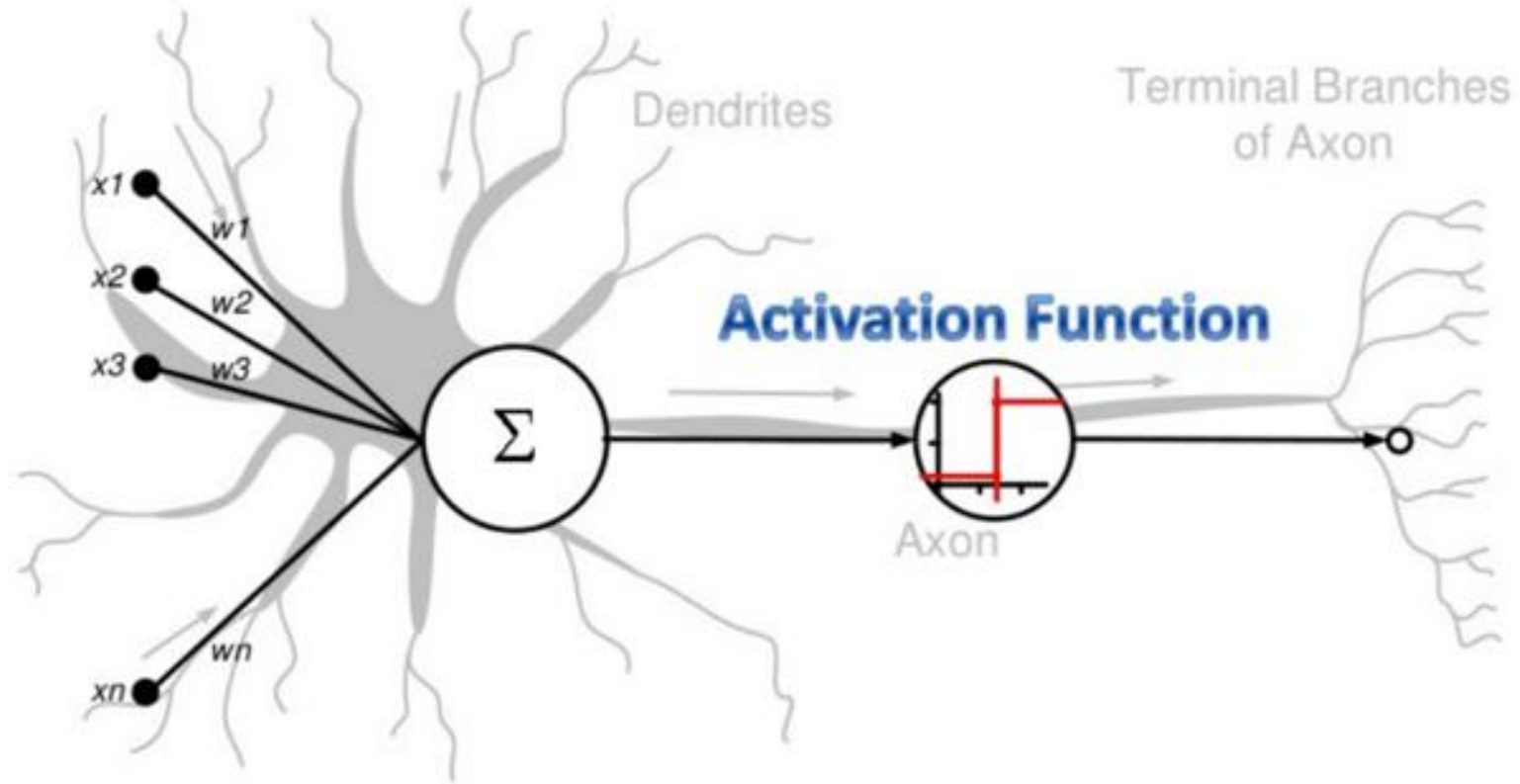


Traditional machine learning uses hand-crafted features, which is tedious and costly to develop.

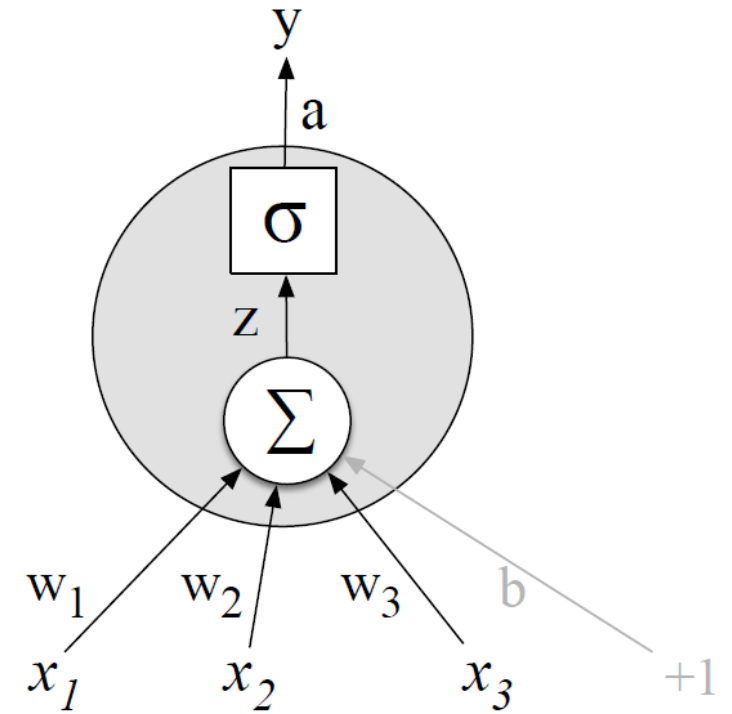


Deep learning learns hierarchical representation from the data itself, and scales with more data.





- 1.Pesos (w):** Controlan la fuerza de la señal de entrada.
- 2.Bias (b):** tiene el efecto de cambiar la función de activación en una cantidad constante.
- 3.Función de activación:** Calcula la salida vía una función no lineal.



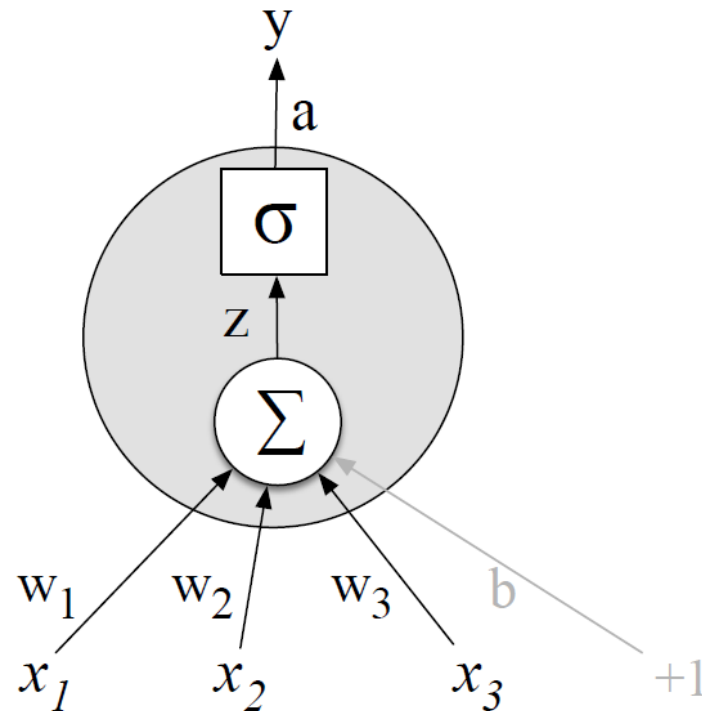
$$w = [0.2, 0.3, 0.9]$$

$$x = [0.5, 0.6, 0.1]$$

$$b = 0.5$$

$$y = \frac{1}{1 + e^{-(.5 \cdot 0.2 + .6 \cdot 0.3 + .1 \cdot 0.9 + .5)}}$$

$$y = \frac{1}{1 + e^{-(0.87)}} = 0.7$$



$$y = \sigma(w \cdot x + b) = \frac{1}{1 + \exp(-(w \cdot x + b))}$$

Esto es algo que ya vimos en esta clase!!!

$$x \longrightarrow \text{Nodo (Neurona)} \longrightarrow a = f(w \cdot x + b)$$

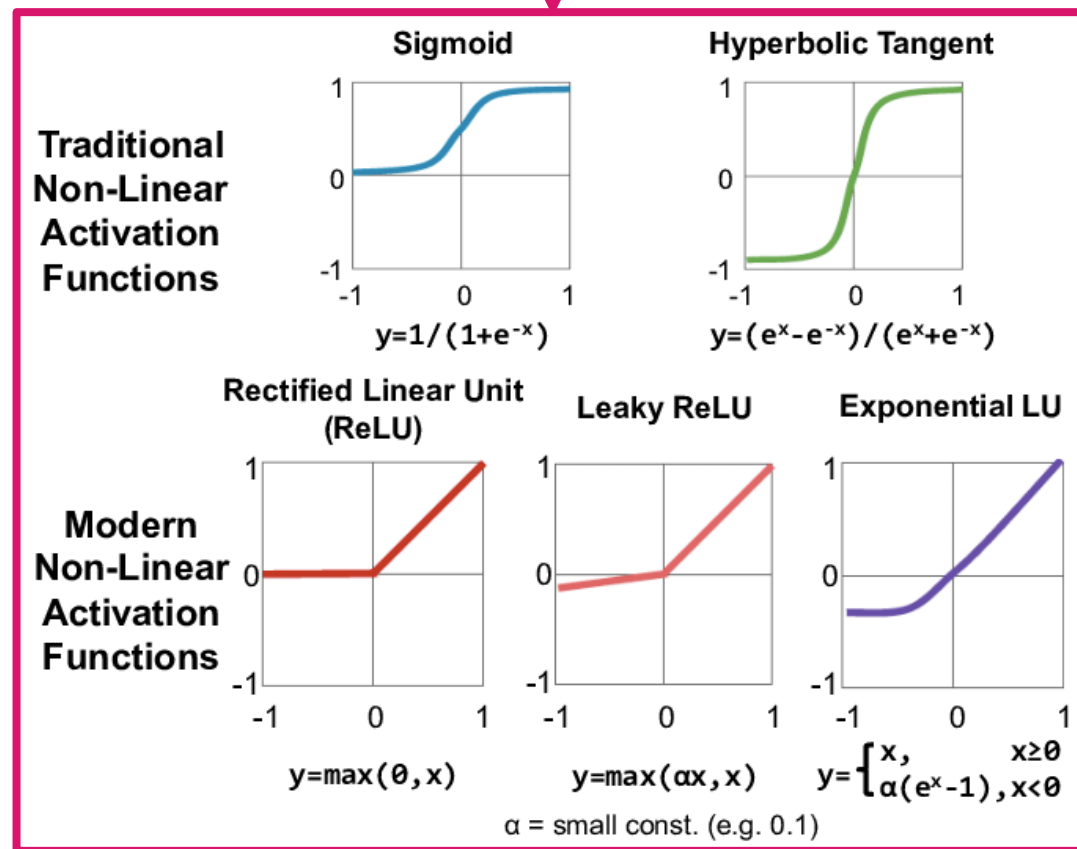


Imagen tomado de Caffe Tutorial

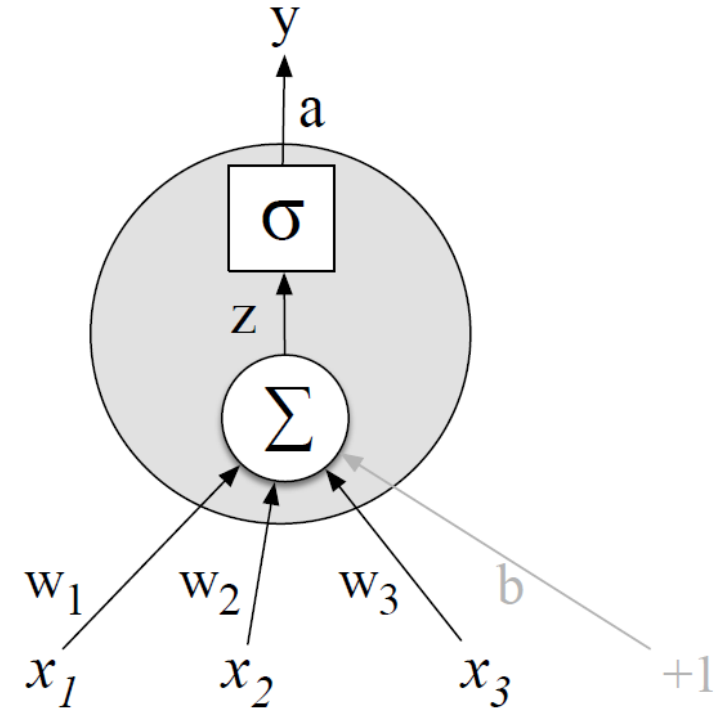
Clasificador binario (+/-) de reviews de películas.
Etiquetas: 0/1

Ingeniería de características:

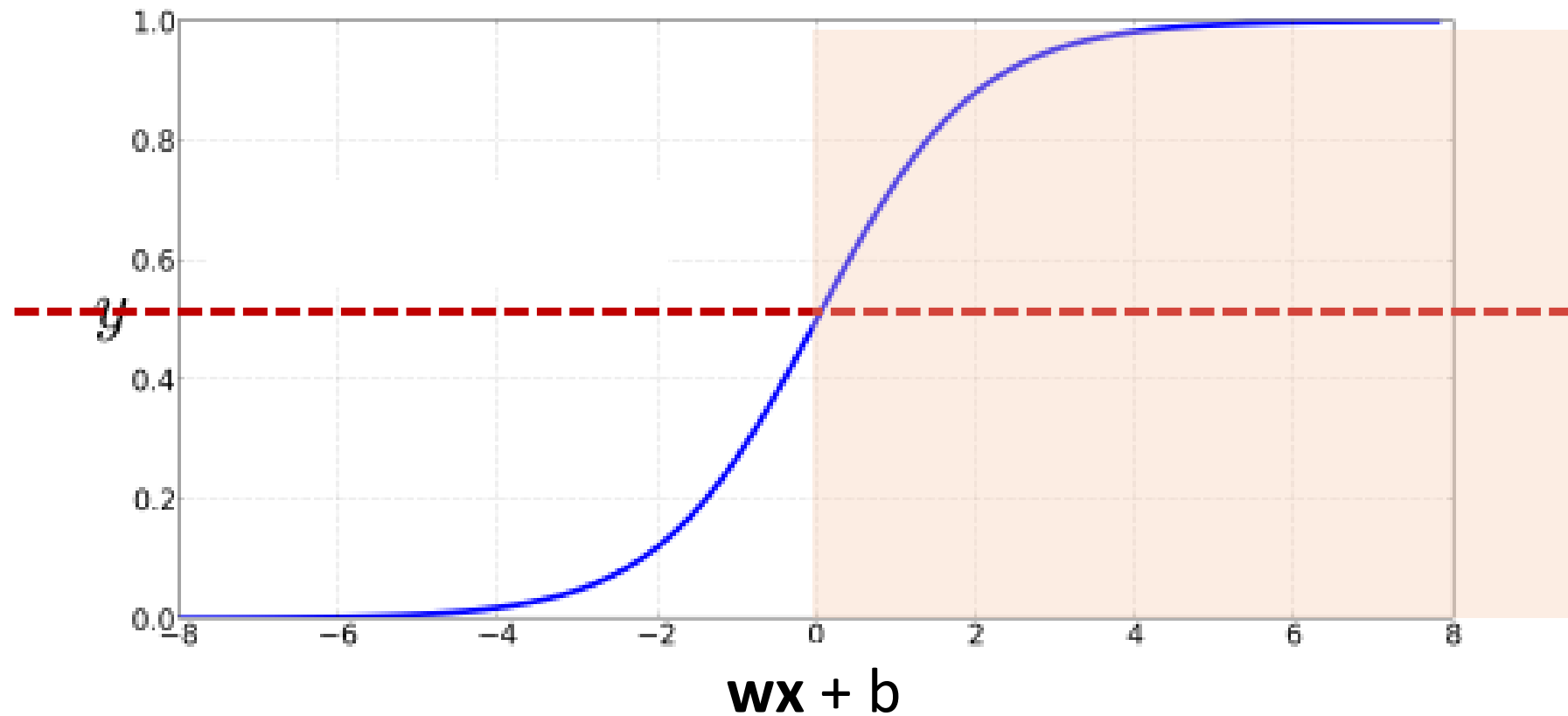
- x_i = "review contiene 'asombrosa'": $w_i = +10$
- x_j = "review contiene 'pésima'": $w_j = -10$
- x_k = "review contiene 'mediocre'": $w_k = -2$
- $b = 11$

El peso me dice la importancia de la característica.

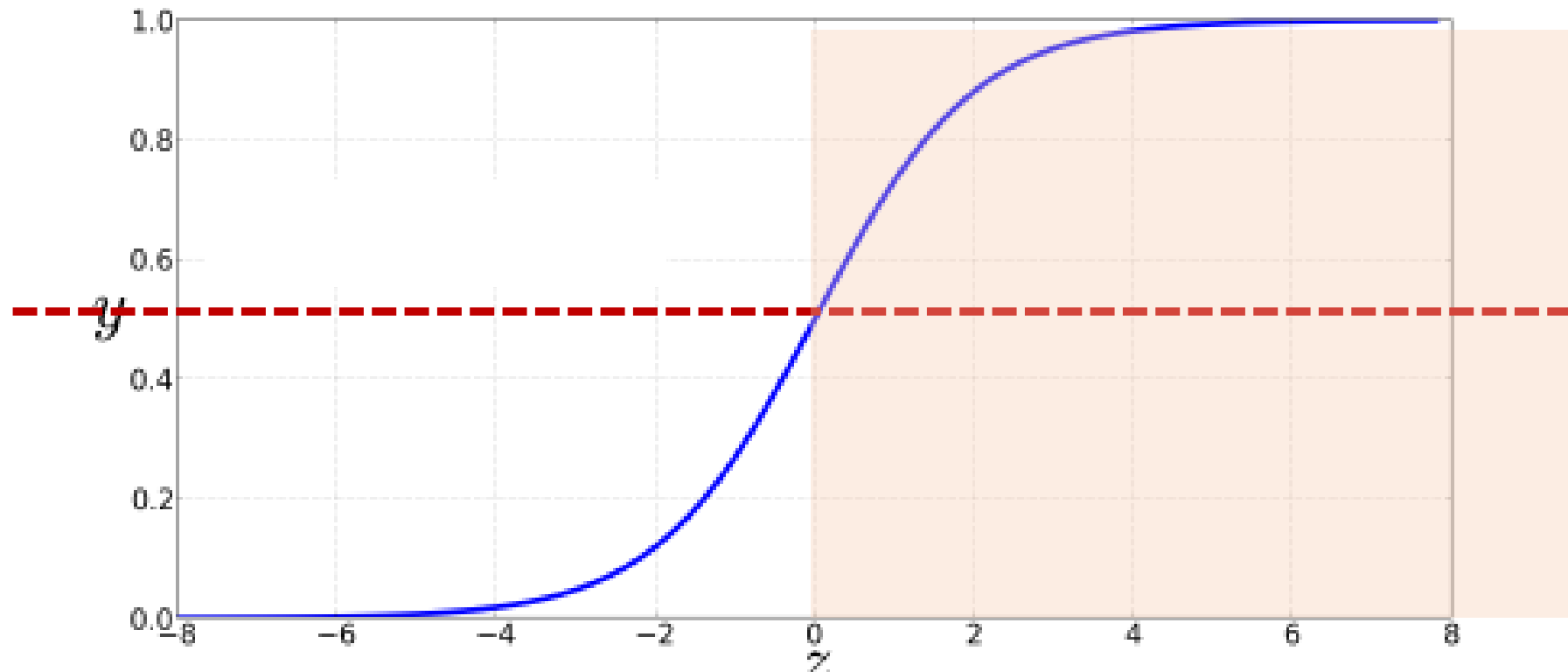
Salida, la predicción de una clase: $\hat{y} \in \{0, 1\}$



$$\sigma(\mathbf{w}\mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}\mathbf{x} + b)}} \text{ lo tratamos como } P(y = 1|\mathbf{x})$$



$$\hat{y} = \begin{cases} 1 & \text{if } P(y = 1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases} \quad \begin{array}{l} \text{if } w \cdot x + b > 0 \\ \text{if } w \cdot x + b \leq 0 \end{array}$$



HAGAMOS UN PEQUEÑO EJERCICIO

- x_i = "review contiene 'asombrosa'": $w_i = +10$
- x_j = "review contiene 'pésima'": $w_j = -10$
- x_k = "review contiene 'mediocre'": $w_k = -2$
- $b = 11$

Es una película pésima y mediocre.

- $x_i = 0$, $w_i = +10$
- $x_j = 1$, $w_j = -10$
- $x_k = 1$, $w_k = -2$

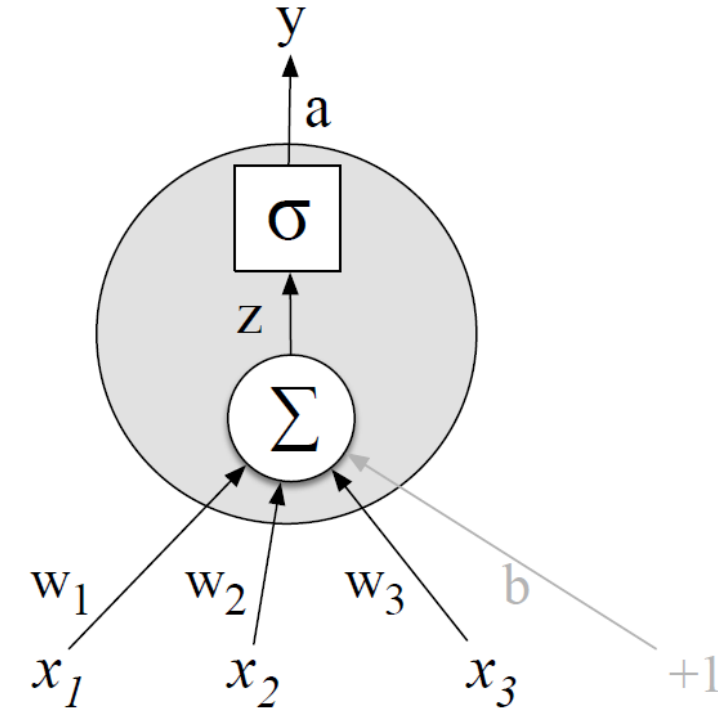
$$\mathbf{x} = [0, 1, 1], \quad \mathbf{w} = [10, -10, -2]$$

$$\hat{y} = \sigma(\mathbf{w}\mathbf{x} + b)$$

$$\hat{y} = \sigma([0, 1, 1][10, -10, -2] + 11)$$

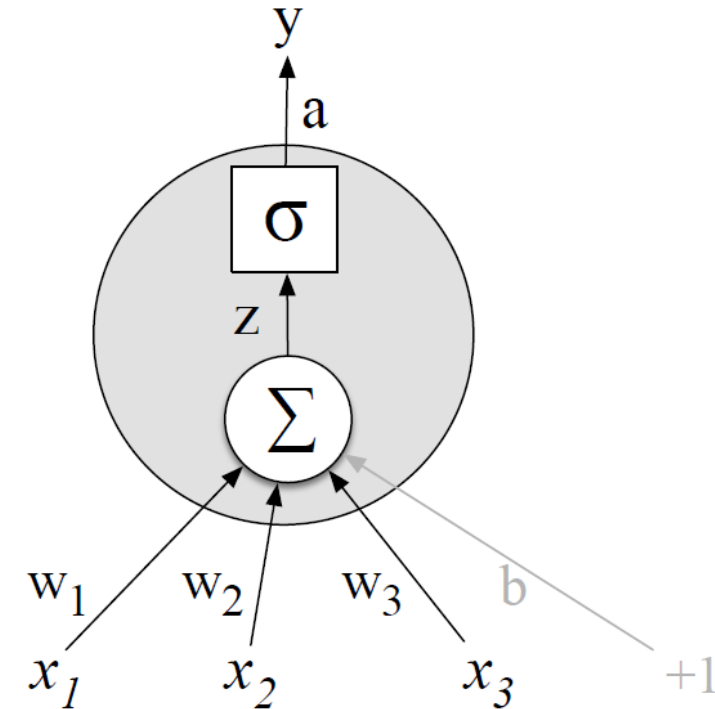
$$\hat{y} = \sigma(-1) = 0.26$$

La predicción es la clase **negativa**



$$\hat{y} = \begin{cases} 1 & \text{if } P(y = 1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

1. Una neurona artificial (nodo) no es otra cosa que una regresión logística con una interpretación biológica.
2. Una función de activación **sigmoideal** me permite interpretar la salida de la red como una probabilidad.
3. La función de activación sigmoideal sirve para problemas de clasificación binarios.
4. Simplificando se puede ver la operación de la red como la multiplicación de dos vectores (**tensores**) entrada*pesos a los cuales se les aplica una función no lineal.
5. El clasificador es bueno (“predice correctamente”) si tiene los pesos y bias adecuados.



De donde vienen los pesos?

Clasificación Supervisada:

- Conocemos el label correcto y para cada ejemplo de entrada x
- La neurona produce una estimación $\hat{y}$

Queremos buscar los w y b que minimizan la distancia entre la realidad y y nuestra estimación $\hat{y}$

- Es necesario un estimador de distancia: **loss function** o **cost function**.
- Necesitamos un algoritmo de optimización que actualice w y b para minimizar la **loss function**.

Componentes del aprendizaje:

- Loss function:
 - **Cross entropy loss**
- Algoritmo de optimización:
 - **Gradiente descendiente estocástico.**

Cross entropy loss:

$$L_{\text{CE}}(\hat{y}, y) = -\log p(y|x) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

$$L_{\text{CE}}(\hat{y}, y) = -[y \log \sigma(w \cdot x + b) + (1 - y) \log(1 - \sigma(w \cdot x + b))]$$

Queremos que el loss sea:

- Pequeño si la estimación del modelo es cercana a la correcta.
- Grande si el modelo es confuso.

$$L_{CE}(\hat{y}, y) = -[y \log \sigma(w \cdot x + b) + (1 - y) \log (1 - \sigma(w \cdot x + b))]$$

Es una película pésima y mediocre.

- $x_i = 0, \quad w_i = +10$
- $x_j = 1, \quad w_j = -10$
- $x_k = 1, \quad w_k = -2$

$$\hat{y} = \sigma(wx + b)$$

$$\hat{y} = \sigma([0, 1, 1] [10, -10, -2] + 11)$$

$$\hat{y} = \sigma(-1) = 0.26$$

La clase real es **negativa** (i.e. $y = 0$)

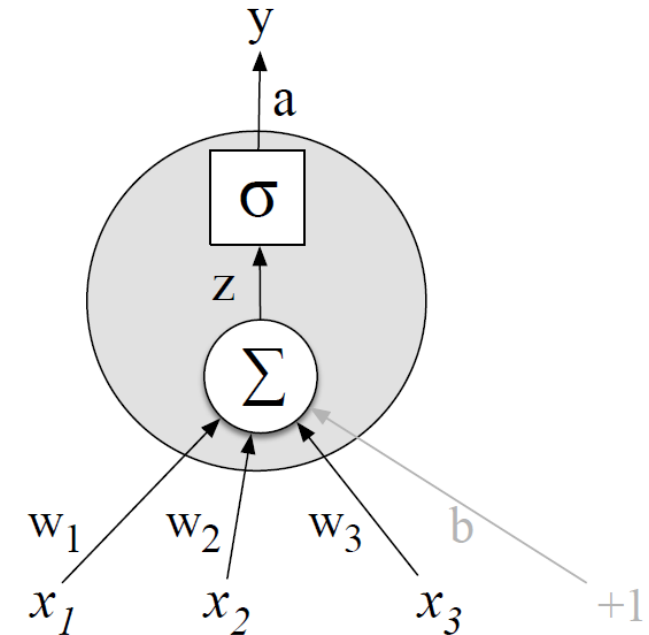
$$L_{CE}(\hat{y}, y) = -[\log(1 - \sigma(wx + b))]$$

$$L_{CE}(\hat{y}, y) = -\log(1 - 0.26) = -\log(0.74) = 0.1307$$

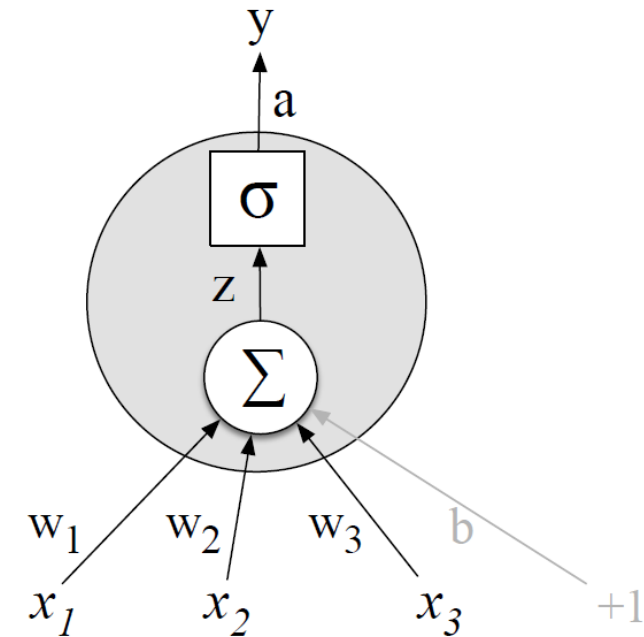
Supongamos por un momento que la clase real es **positiva** (i.e. $y = 1$)

$$L_{CE}(\hat{y}, y) = -[\log(\sigma(wx + b))]$$

$$L_{CE}(\hat{y}, y) = -\log(0.26) = -\log(0.74) = 0.58$$



1. Métrica que nos permite determinar que tan cercana a lo deseable/realidad son las predicciones de la neurona.
2. Debe ser fácilmente derivable lo cual es deseable para algoritmos de optimización basados en gradientes.
3. El **cross-entropy loss** (**log loss**) y sus derivados es de lejos la función de loss mas utilizada en Deep Learning para tareas de clasificación.
4. El **MSE** (mean squared error) y sus derivados es de lejos la función de loss mas utilizada en Deep Learning para tareas de regresión.



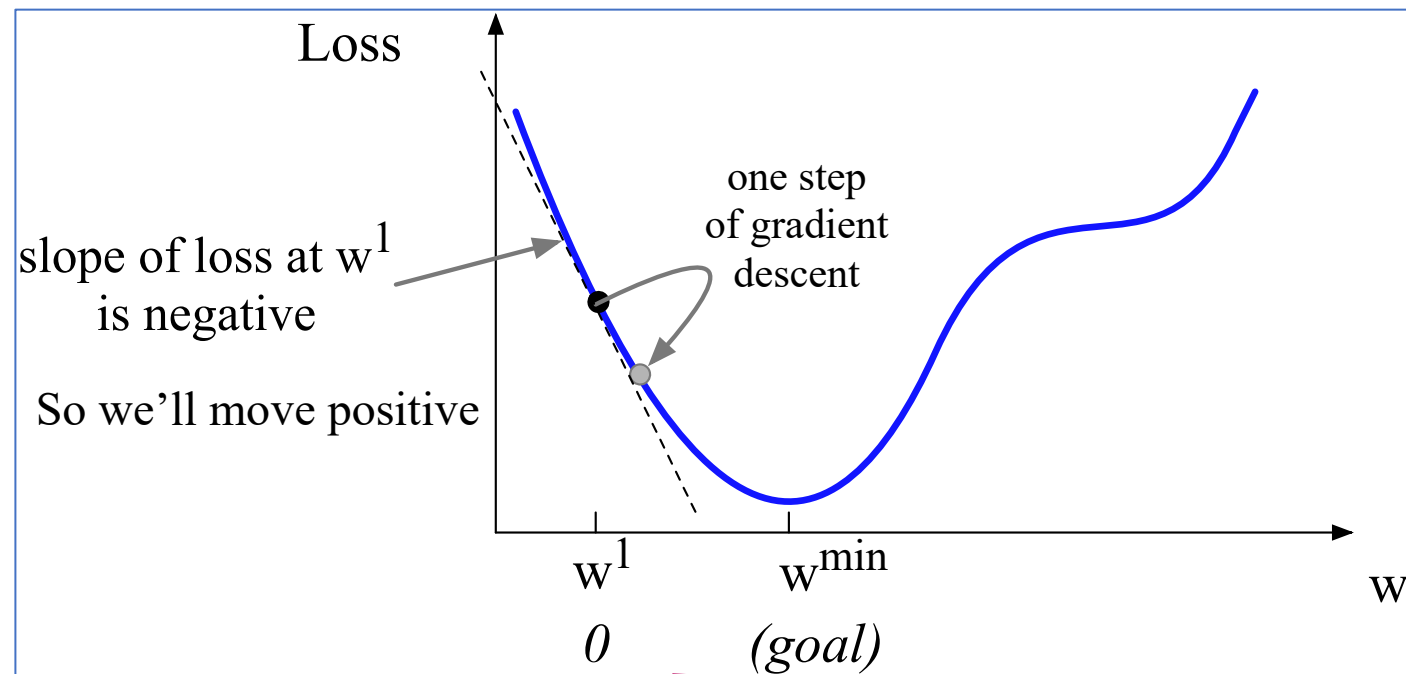
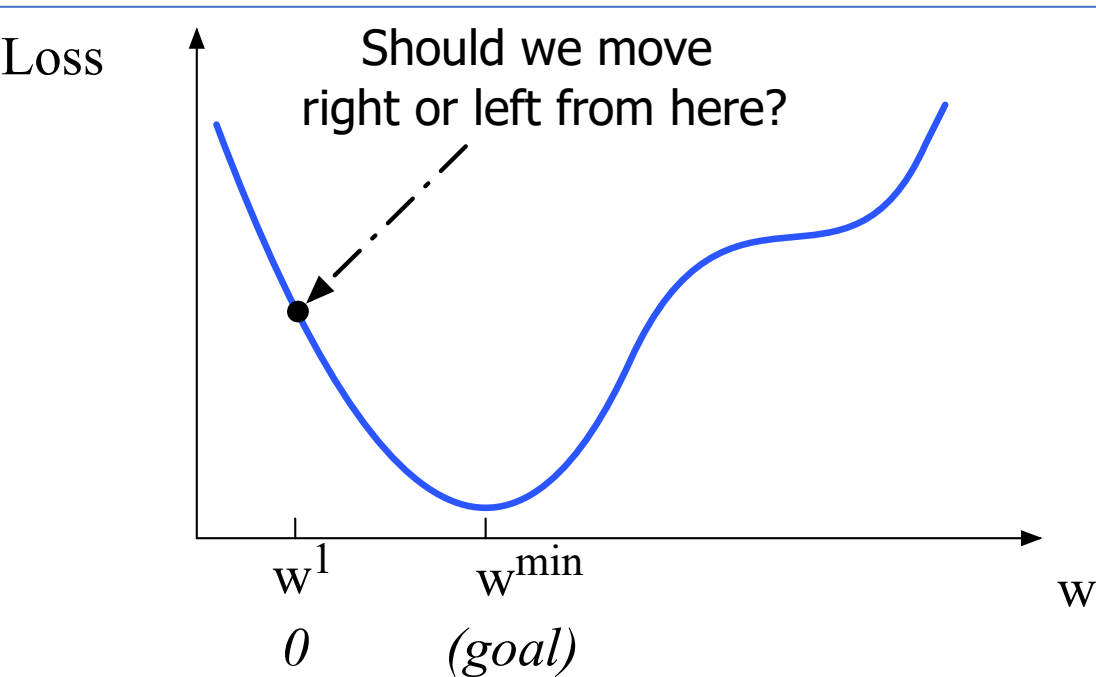
Objetivo: Minimizar el loss!!!

Vamos a dejar explicito que la función de loss esta parametrizada por los pesos:

- $\theta = (\mathbf{w}, b)$
- $\hat{y} = f(x, \theta)$

Queremos los pesos que minimizan el loss, promediados sobre todos los ejemplos:

$$\hat{\theta} = \operatorname{argmin}_{\theta} \frac{1}{m} \sum_{i=1}^m L_{\text{CE}}(f(x^{(i)}; \theta), y^{(i)})$$



Para un solo nodo (neurona) la función de loss es convexa
Una función convexa tiene solo un mínimo.

Ok ya se en que dirección, pero que tanto muevo los parametros?

Dirección: $-\frac{d}{dw} L(f(x; w), y)$

Que tanto me muevo?

$$w^{t+1} = w^t - \eta \frac{d}{dw} L(f(x; w), y)$$

Tasa de aprendizaje

function STOCHASTIC GRADIENT DESCENT($L()$, $f()$, x , y) **returns** θ

where: L is the loss function

f is a function parameterized by θ

x is the set of training inputs $x^{(1)}, x^{(2)}, \dots, x^{(m)}$

y is the set of training outputs (labels) $y^{(1)}, y^{(2)}, \dots, y^{(m)}$

$\theta \leftarrow 0$

repeat til done # see caption

For each training tuple $(x^{(i)}, y^{(i)})$ (in random order)

1. Optional (for reporting):

 Compute $\hat{y}^{(i)} = f(x^{(i)}; \theta)$

 Compute the loss $L(\hat{y}^{(i)}, y^{(i)})$

2. $g \leftarrow \nabla_{\theta} L(f(x^{(i)}; \theta), y^{(i)})$

3. $\theta \leftarrow \theta - \eta g$

return θ

Solo tiene sentido con una sola neurona.

Entrenamiento en Batch. Mini-batch training (512,1024).

Por eso se llama estocástico.

Se definen un número de épocas – época es un completo por todos los ejemplos de entrenamiento.

EJEMPLO: GRADIENTE DESCENDIENTE ESTOCÁSTICO

Imaginen un clasificador para identificar si se le da el crédito a una persona o no.

Ingeniería de características:

$x_1 = 3$ (*Número de hijos*)

$x_2 = 2$ (*Años de continuidad en el empleo*)

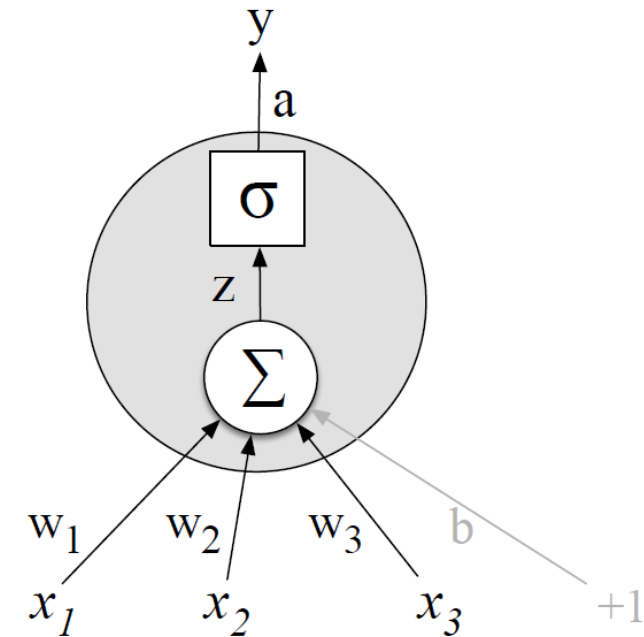
$y = 1$ (*Salida real: si le dieron el credito*)

Parámetros

$w_1 = w_2 = b = 0$

$\eta = 0.1$

Haga una ronda de actualización de pesos



EJEMPLO: GRADIENTE DESCENDIENTE ESTOCÁSTICO

$$w^{t+1} = w^t - \eta \frac{d}{dw} L(f(x; w), y)$$

Optimización por gradiente descendiente

$$\frac{d}{dw_j} L_{CE}(f(\mathbf{x}; \mathbf{w}), y) = (\hat{y} - y)x_j = (\sigma(\mathbf{w}\mathbf{x} + b) - y)x_j$$

Derivada del CE

El gradiente va tener en realidad 3 dimensiones (un valor para cada parámetro)

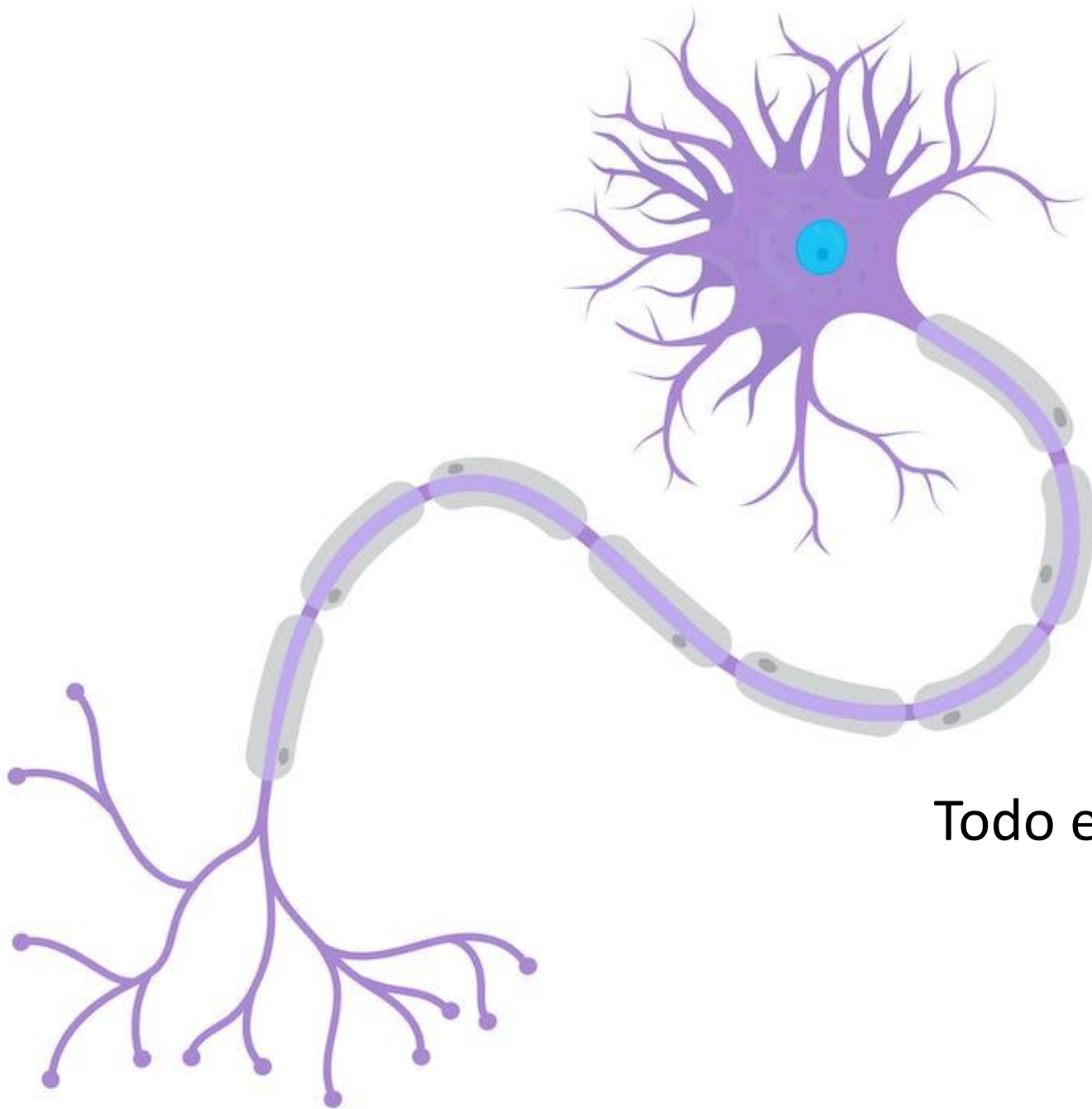
$$\nabla_{w,b} = \begin{bmatrix} \frac{\partial L_{CE}(\hat{y}, y)}{\partial w_1} \\ \frac{\partial L_{CE}(\hat{y}, y)}{\partial w_2} \\ \frac{\partial L_{CE}(\hat{y}, y)}{\partial b} \end{bmatrix} = \begin{bmatrix} (\sigma(w \cdot x + b) - y)x_1 \\ (\sigma(w \cdot x + b) - y)x_2 \\ \sigma(w \cdot x + b) - y \end{bmatrix} = \begin{bmatrix} (\sigma(0) - 1)x_1 \\ (\sigma(0) - 1)x_2 \\ \sigma(0) - 1 \end{bmatrix} = \begin{bmatrix} -0.5x_1 \\ -0.5x_2 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}$$

EJEMPLO: GRADIENTE DESCENDIENTE ESTOCÁSTICO

Ya tenemos el gradiente actualicemos los pesos:

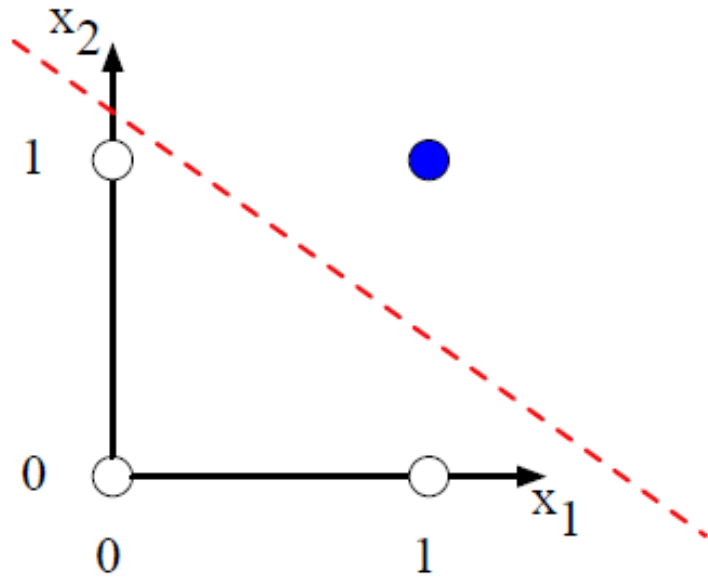
$$w^{t+1} = w^t - \eta \frac{d}{dw} L(f(x; w), y)$$

$$w^{t+1} = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix} - \eta \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix} = \begin{bmatrix} .15 \\ .1 \\ .05 \end{bmatrix}$$

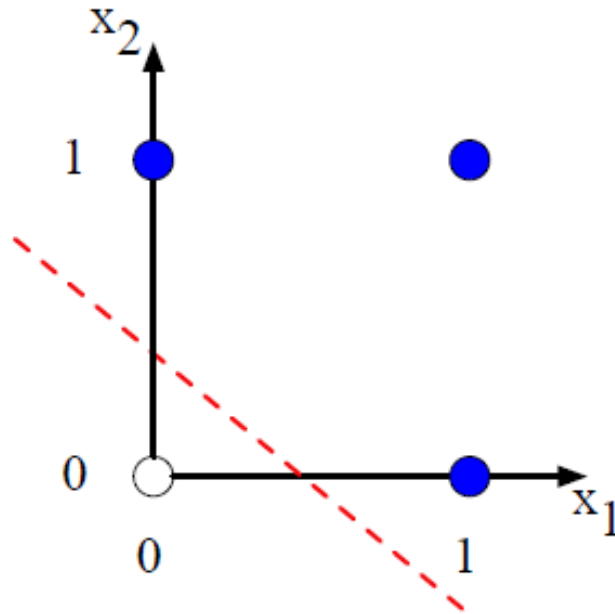


Todo esto fue para una sola neurona!!!

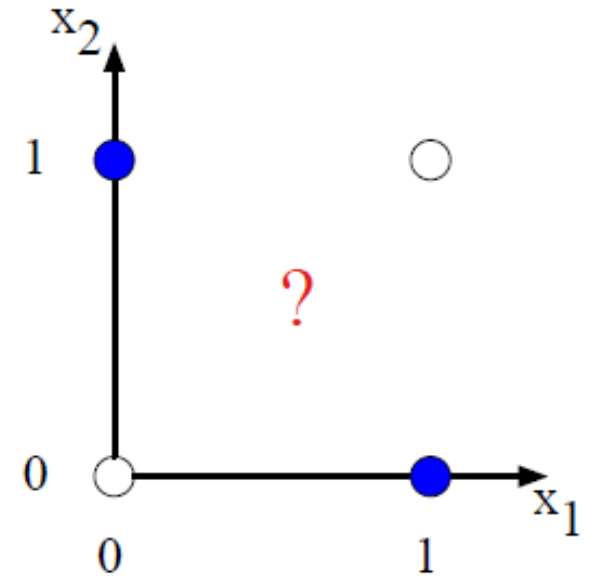
Una sola neurona puede no ser suficiente para problemas no-lineales



a) x_1 AND x_2



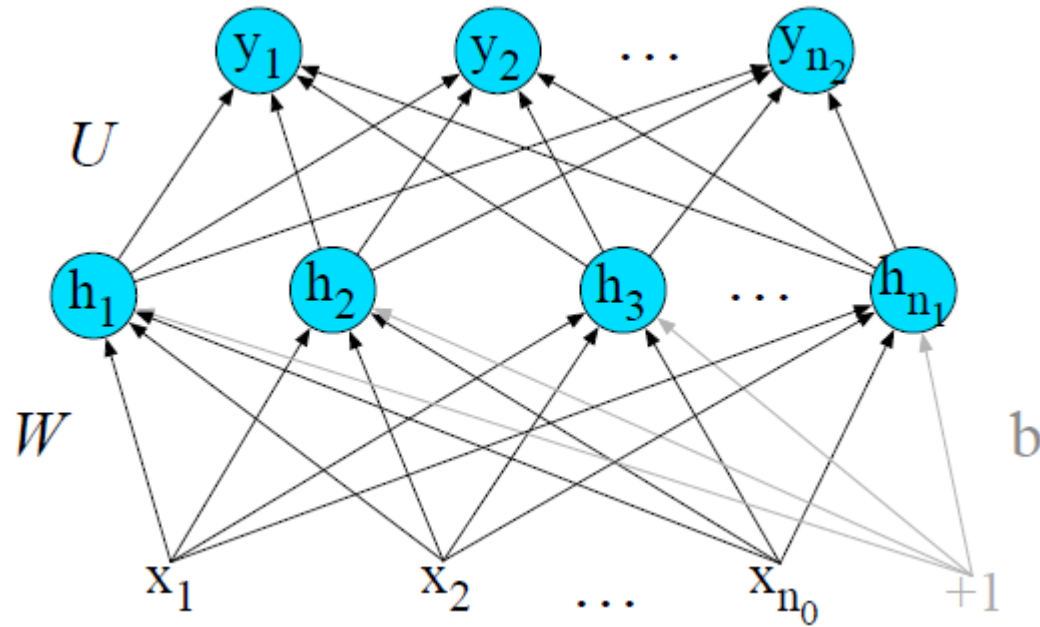
b) x_1 OR x_2



c) x_1 XOR x_2

Solución: Pues conectemos múltiples neuronas!!!

Redes Neuronales: combinación de Neuronas Feed-Forward (MLP)

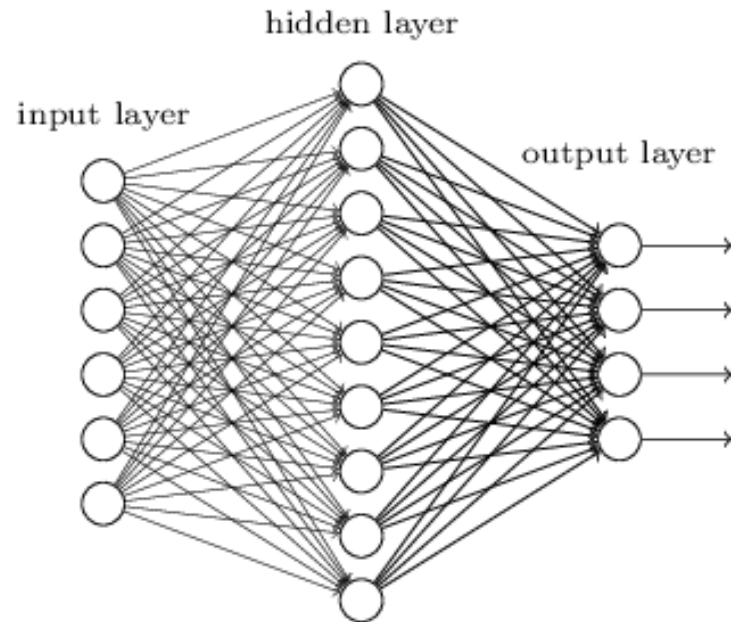


- Las salidas de cada unidad/nodo son pasados a los nodos de la siguiente capa.
- Normalmente totalmente conectada.
- Cada unidad/nodo conectada sin ciclos.
- Cada unidad/nodo tiene sus parámetros w y b .

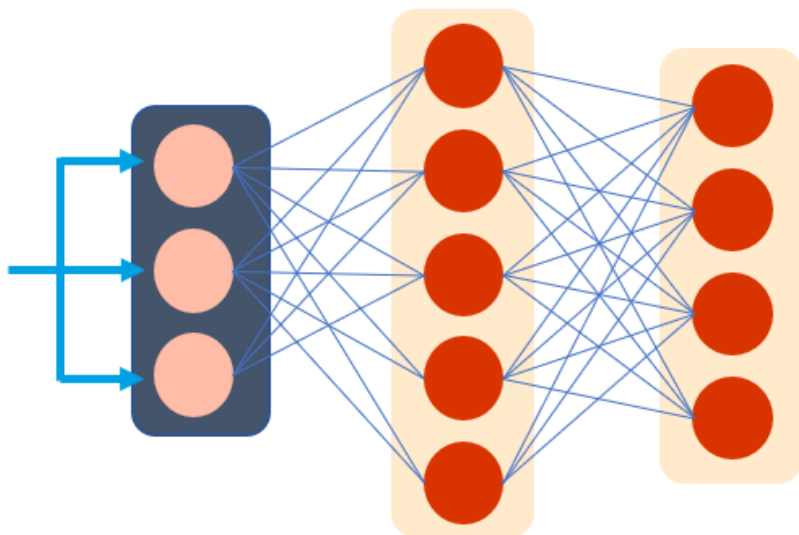
Redes Neuronales: combinación de Neuronas

Ojo con los dibujos...

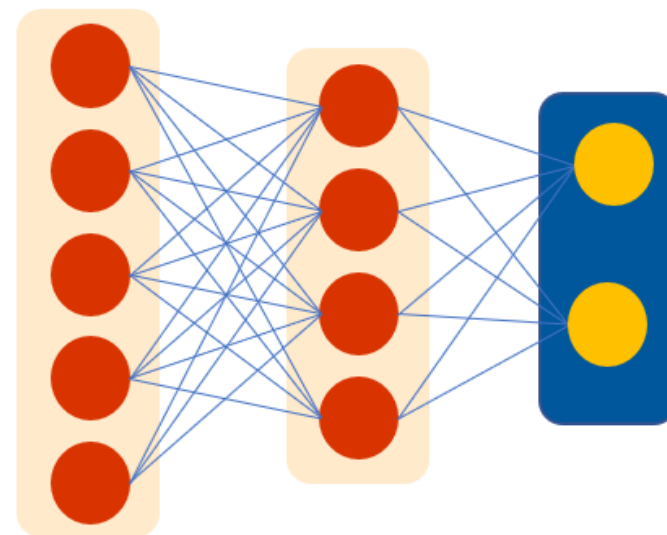
"Non-deep" feedforward
neural network



Forward



Muchas
capas
ocultas
intermedias
.....



Backpropagation

Cada uno de los pesos de la red neuronal recibe una actualización proporcional a la derivada parcial de la función de error con respecto al peso actual en cada iteración del entrenamiento.

PROBLEMA DE GRADIENTE DESVANECIENTE

Datos



Ubuntu Dialogue Corpus (2016) –
1 millón de diálogos.

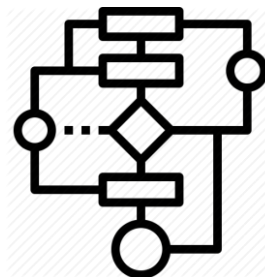
BlenderBot (2020) – 9.4 billón
de parámetros – 1.5 billones de
conversaciones Reddit.

Hardware



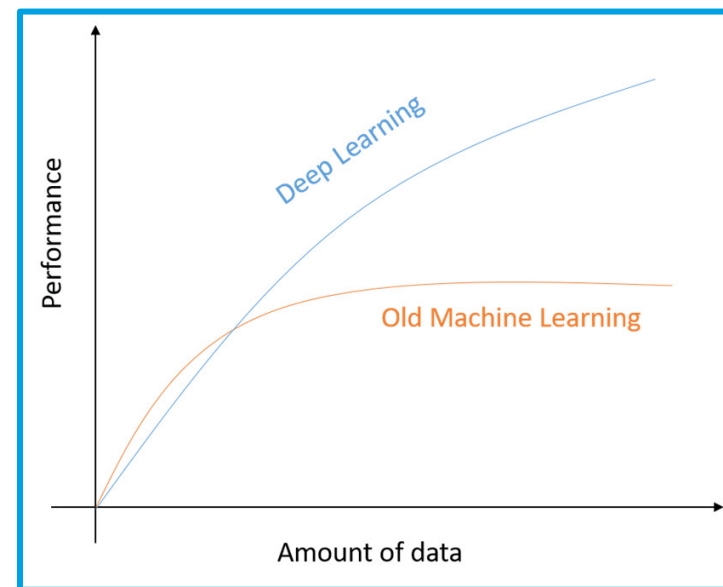
Las GPUs tienen la
capacidad de hacer varias
tareas de forma
simultánea.

Algoritmos



Algoritmos e
implementaciones de estos
mas eficientes, por ejemplo
la ReLU.

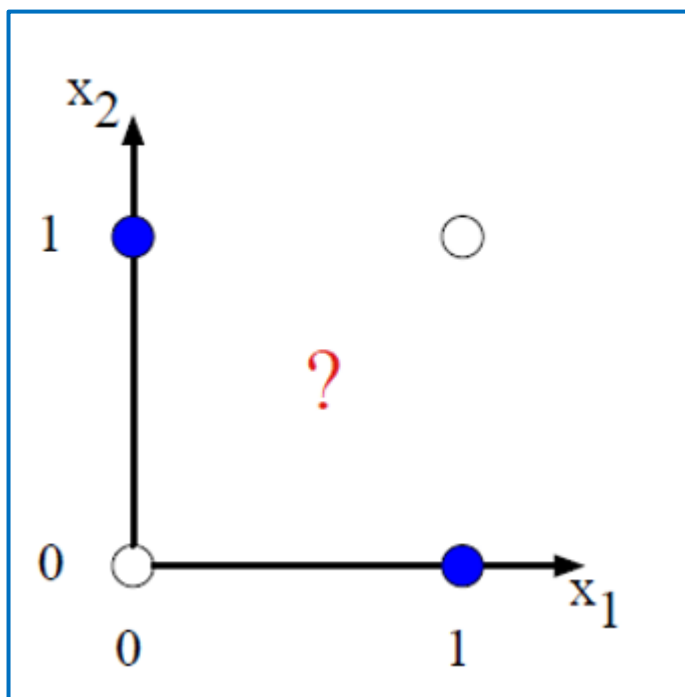
Leyes de crecimiento



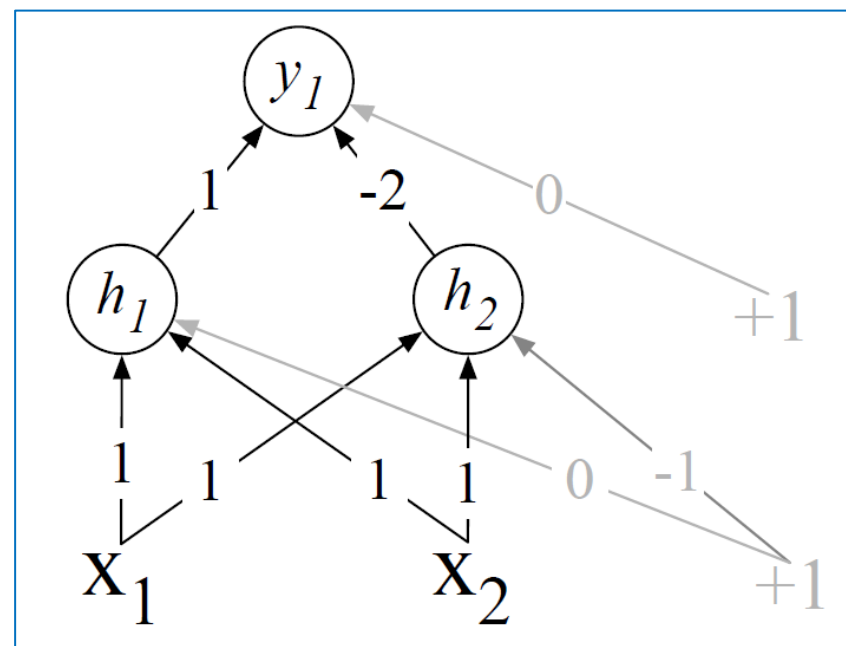
REDES PROFUNDAS

CAPAS OCULTAS: NUEVAS REPRESENTACIONES DE LAS ENTRADAS

Ok entiendo porque las redes profundas en la actualidad tienen sentido, pero al principio de esta sesión hablamos que **eran capaces de aprender por su cuenta características!!**



Solución



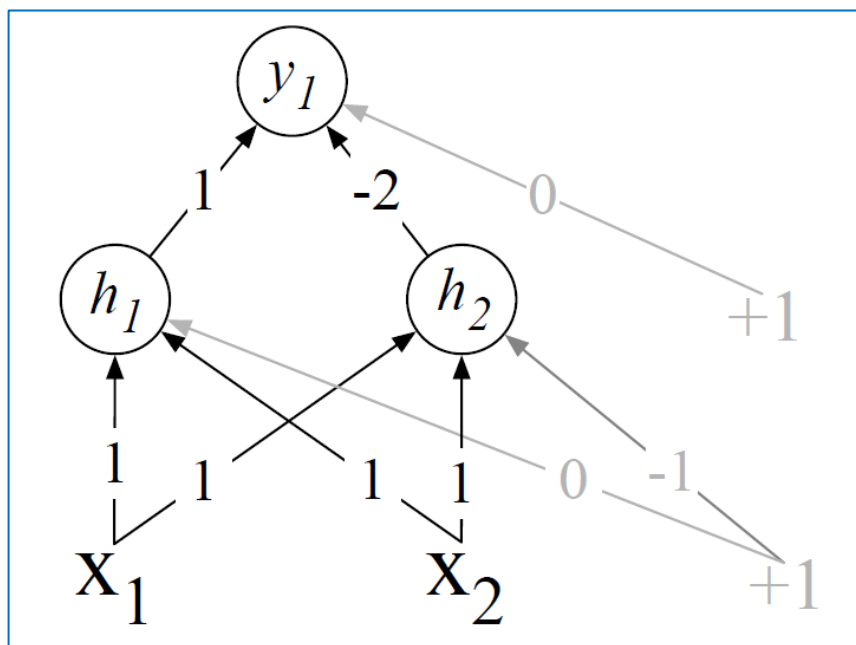
3 Unidades: ReLU.

2 Unidades en la capa oculta ($h_1 + h_2$)

1 Unidad de salida (y)

REDES PROFUNDAS

CAPAS OCULTAS: NUEVAS REPRESENTACIONES DE LAS ENTRADAS



3 Unidades: ReLU.

2 Unidades en la capa oculta ($h_1 + h_2$)

1 Unidad de salida (y)

$x = [0, 0]$

$y_1 = ?$

$h_1 = \max(0, 0)$

$h_2 = \max(0, -1)$

$h = [0, 0]$

$y_1 = \max(0, 0) = 0$

$x = [1, 0]$

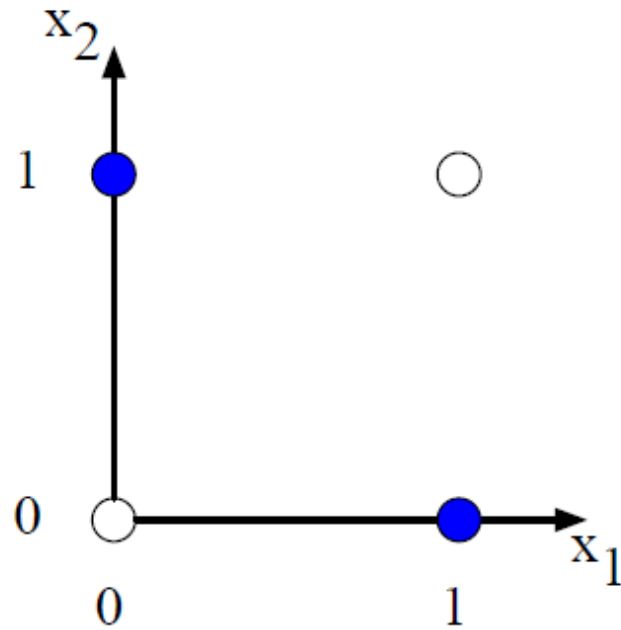
$h_1 = \max(0, 1) = 1$,

$h_2 = \max(0, 0) = 0$,

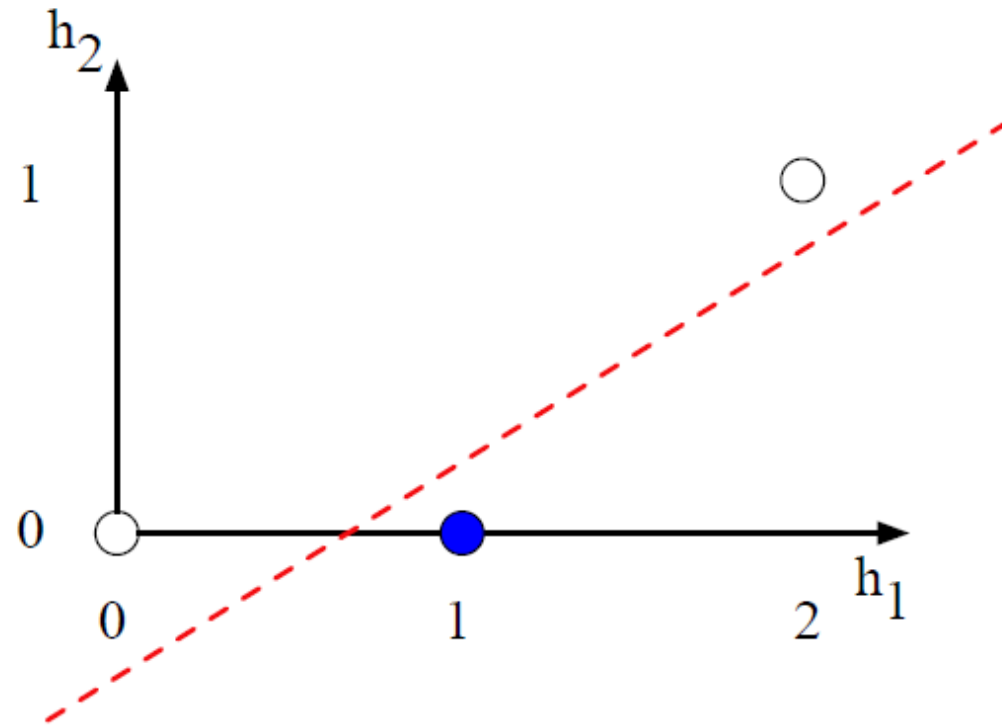
$y_1 = \max(0, 1) = 1$

x_1	x_2	Salida
0	0	0
0	1	1
1	0	1
1	1	0

Capas Ocultas: Nueva representación de las entradas.



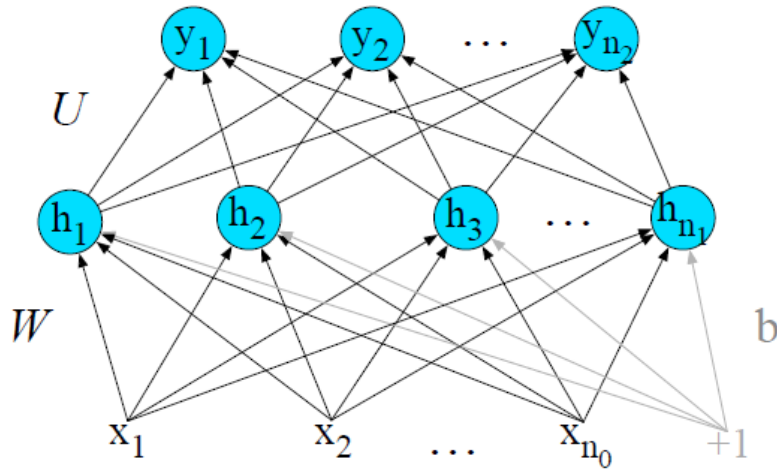
a) The original x space



b) The new (linearly separable) h space

Nuevas representaciones de las entradas $\approx$ Genera las features

REDES PROFUNDAS NOTACIÓN CAPAS OCULTAS

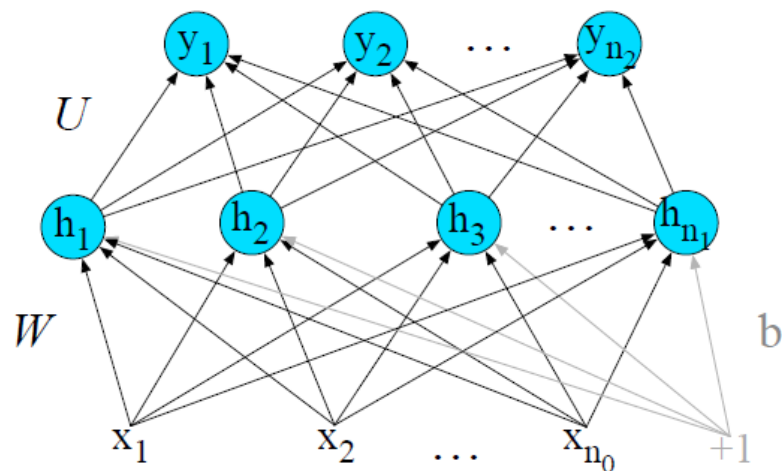


- Representamos los parámetros de una capa oculta como la combinación del vector de pesos (w_i) y el bias (b_i) de cada unidad i mediante una matriz de pesos $\mathbf{W}$ y un vector de bias $\mathbf{b}$.
- Cada elemento $\mathbf{W}_{ij}$ representa el peso de la conexión de la i th entrada x_i a la unidad/nodo $\mathbf{h}_j$.
- La salida de la capa oculta el vector $\mathbf{h}$ es entonces calculado como:

$$\mathbf{h} = \underbrace{\sigma(\underbrace{\mathbf{W}\mathbf{x} + \mathbf{b}}_{\text{vector}})}_{\text{vector}}$$

La función de activación es aplicada a cada elemento del vector.

REDES PROFUNDAS NOTACIÓN CAPAS OCULTAS

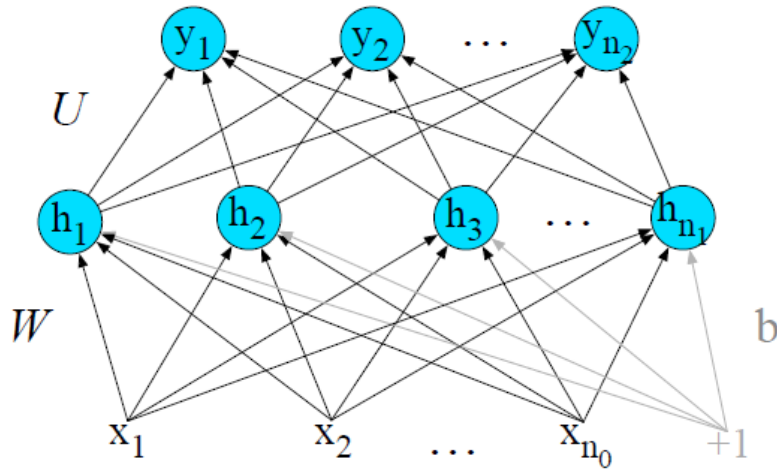


- El vector de entradas $\mathbf{x}$ es de n_0 dimensiones (0 denota la capa de entrada): $\mathbf{x} \in \mathbb{R}^{n_0}$ (vector columna $[n_0, 1]$).
- La capa oculta tiene dimensionalidad n_1 por lo tanto $\mathbf{h} \in \mathbb{R}^{n_1}$ y $\mathbf{b} \in \mathbb{R}^{n_1}$.
- La matriz de pesos $\mathbf{W}$ por lo tanto: $\mathbf{W} \in \mathbb{R}^{n_1 \times n_0}$ ($[n_1, n_0]$).

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$$

$$\left(\begin{pmatrix} w_{1,1} & \cdots & w_{1,n_0} \\ \vdots & \ddots & \vdots \\ w_{n_1,1} & \cdots & w_{n_1,n_0} \end{pmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_{n_0} \end{bmatrix} + \begin{bmatrix} b_1 \\ \vdots \\ b_{n_1} \end{bmatrix} \right) = \begin{bmatrix} w_{1,1}x_1 + \cdots + w_{1,n_0}x_{n_0} + b_1 \\ \vdots \\ w_{n_1,1}x_1 + \cdots + w_{n_1,n_0}x_{n_0} + b_{n_1} \end{bmatrix}$$

REDES PROFUNDAS CAPA DE SALIDA



La capa de salida puede o no estar con el parámetro bias, pero si tiene pesos. Denotaremos los pesos de la capa de salida como la matriz $U \in \mathbb{R}^{n_2 \times n_1}$, la salida $z \in \mathbb{R}^{n_2}$ se calcula como:

$$z = Uh$$

Sin embargo, z no puede ser la salida del clasificador, ya que es un vector de valores reales números, mientras que lo que necesitamos para la clasificación es un vector de probabilidades. **Sigmoidal** para el caso binario y cuando son múltiples clases.

Softmax: codifica una distribución de probabilidad.

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Torna un vector z de K valores arbitrarios en probabilidades (la suma de sus valores es 1):

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

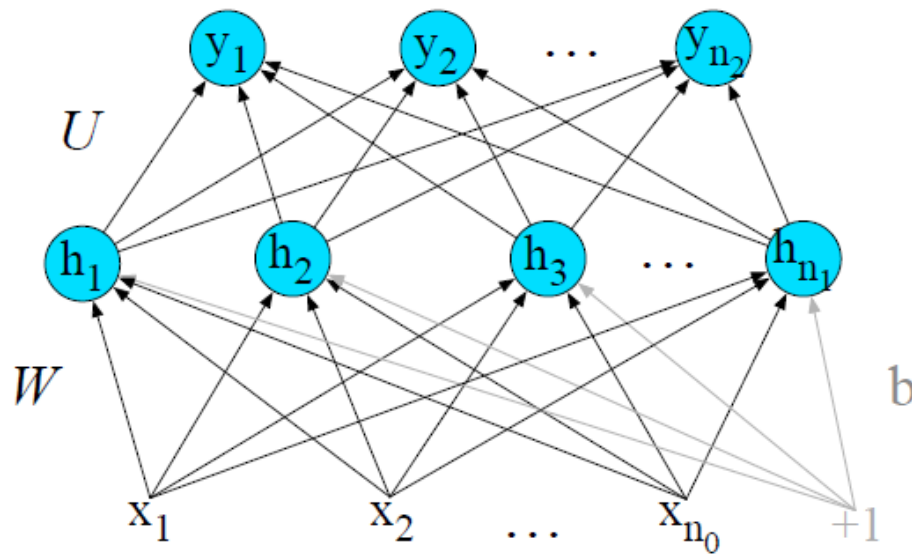
$$z = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$$

$$\text{softmax}(z) = \left[\frac{\exp(z_1)}{\sum_{i=1}^k \exp(z_i)}, \frac{\exp(z_2)}{\sum_{i=1}^k \exp(z_i)}, \dots, \frac{\exp(z_k)}{\sum_{i=1}^k \exp(z_i)} \right]$$

$$\text{softmax}(z) = [0.055, 0.090, 0.0067, 0.10, 0.74, 0.010]$$

REDES PROFUNDAS

NOTACIÓN RESUMEN 2-CAPAS OCULTAS



$$h = \sigma(Wx + b)$$

$$z = Uh$$

$$y = \textit{softmax}(z)$$

- Usaremos superíndices entre corchetes para indicar los números de capa, comenzando en 0 para la capa de entrada.
 - Entonces, $\mathbf{W}^{[1]}$ y $\mathbf{b}^{[1]}$ significará la matriz de pesos y el vector de bias para la primera capa oculta.
 - n_j será el número de unidades/nodos de la capa j .
 - Vamos a usar $g()$ para referirnos a la función de activación.
 - ReLU or tanh para las capas intermedias (ocultas).
 - Softmax para las capas de salida.
 - $\mathbf{a}^{[i]}$ significa la salida de la capa i .
 - $\mathbf{z}^{[i]}$ es la combinación lineal de pesos, entradas y bias: $\mathbf{z}^{[i]} = \mathbf{W}^{[i]} \mathbf{a}^{[i-1]} + \mathbf{b}^{[i]}$
 - $\mathbf{a}^{[i]} = g^{[1]}(\mathbf{z}^{[i]})$
 - El vector de entrada $\mathbf{x} = \mathbf{a}^{[0]}$

- $\mathbf{z}^{[1]} = \mathbf{W}^{[1]} \mathbf{a}^{[0]} + \mathbf{b}^{[1]}$
- $\mathbf{a}^{[1]} = g^{[1]}(\mathbf{z}^{[1]})$
- $\mathbf{z}^{[2]} = \mathbf{W}^{[2]} \mathbf{a}^{[1]} + \mathbf{b}^{[2]}$
- $\mathbf{a}^{[2]} = g^{[2]}(\mathbf{z}^{[2]})$
- $\hat{y} = \mathbf{a}^{[2]}$

Forward Step (Generic Computation)

Given $\mathbf{a}^{[0]}$

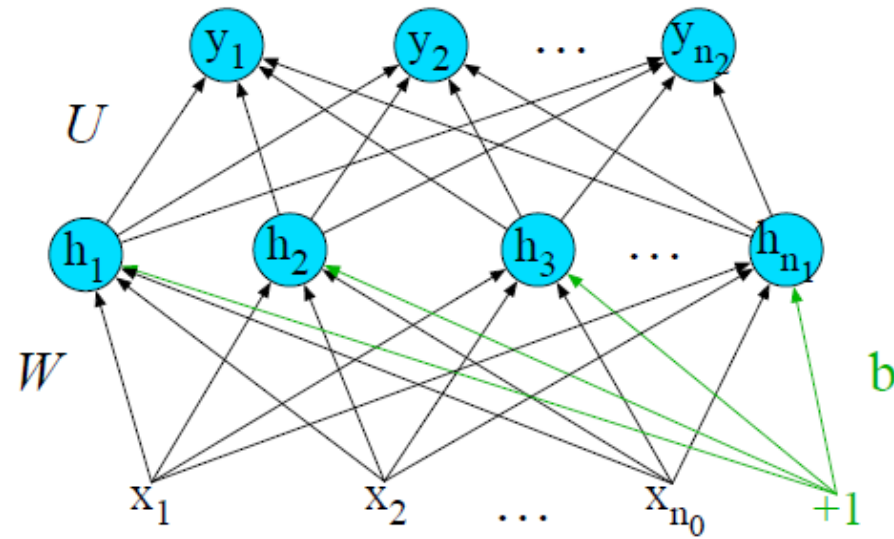
For i in $1..n$:

$$\mathbf{z}^{[i]} = \mathbf{W}^{[i]} \mathbf{a}^{[i-1]} + \mathbf{b}^{[i]}$$

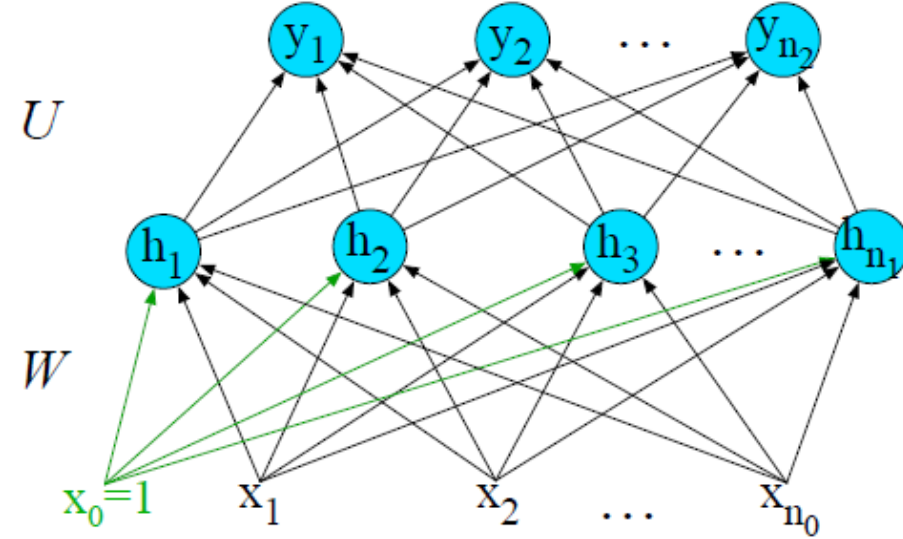
$$\mathbf{a}^{[i]} = g^{[i]}(\mathbf{z}^{[i]})$$

$$\hat{y} = \mathbf{a}^{[n]}$$

REDES PROFUNDAS SIMPLIFICACIÓN DEL BIAS



(a)



(b)

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \text{ se reemplaza por } \mathbf{h} = \sigma(\mathbf{W}\mathbf{x})$$

Entrenamiento de Redes Neuronales
¿Es igual que para una Neurona?

$$\frac{\partial L_{CE}(w, b)}{\partial w_j} = (\hat{y} - y)x_j = (\sigma(wx + b) - y)x_j$$

Derivada cross-entropy loss con función de activación sigmoial

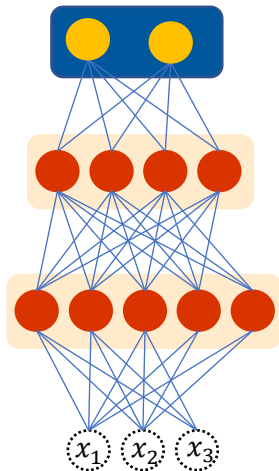
$$\frac{\partial L_{CE}(w, b)}{\partial w_j} = \left(\mathbf{1}\{y = k\} - \frac{e^{(w_k x + b_k)}}{\sum_{j=1}^K e^{(w_j x + b_j)}} \right) x_j$$

Derivada cross-entropy loss con función de activación softmax

Capa de
Salida

Capas
Ocultas

Entradas

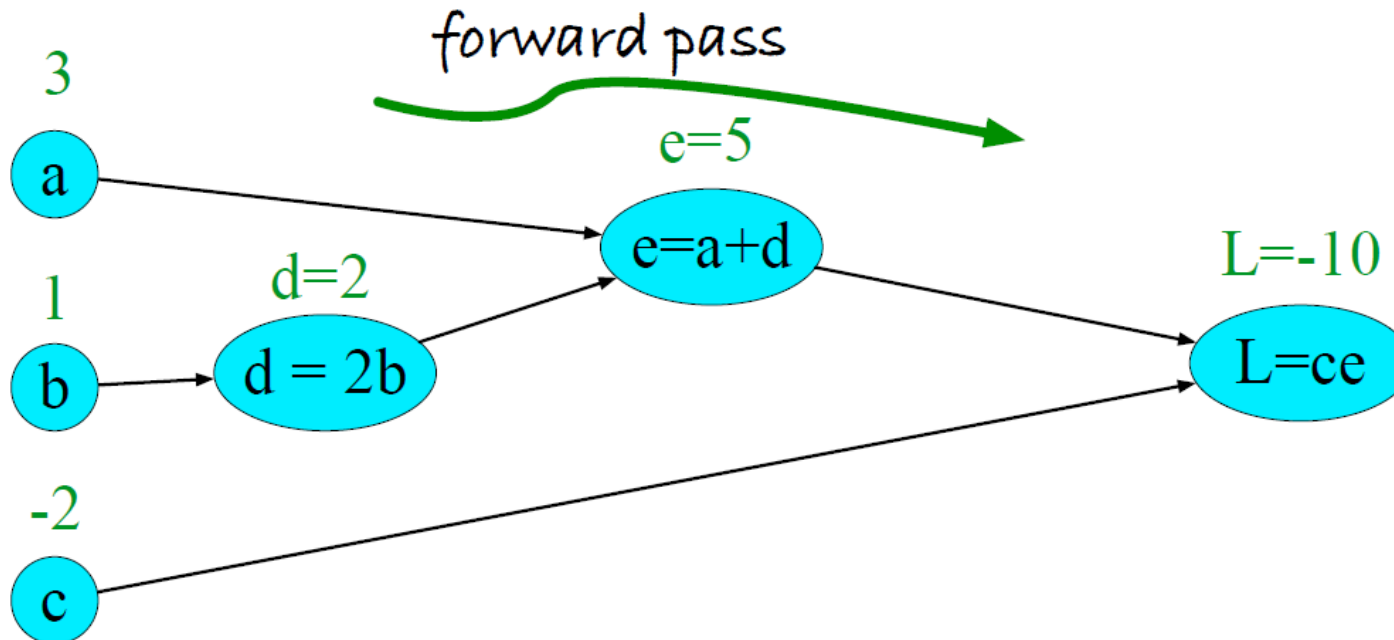


Las anteriores derivadas funcionan para actualizar los pesos de que capa?

Solución: Backprop (1986, Rumelhart).

Backward differentiation
Computation Graphs

- Un grafo de computo es una representación del proceso de cálculo de una expresión, en la que el cálculo se divide en operaciones separadas, cada de los cuales se modela como un nodo en un gráfico.
 - Considere la función **$L(a,b,c) = c(a+2b)$**
 - Suponga que $a=3$, $b=1$, y $c=-2$. Cual es el valor de L ?



GRAFOS DE COMPUTACIÓN

BACKWARD DIFFERENTIATION

- El objetivo es calcular la derivada de la función L con respecto a cada una de esas variables

$$\frac{\partial L}{\partial a}, \frac{\partial L}{\partial b}, \frac{\partial L}{\partial c}.$$

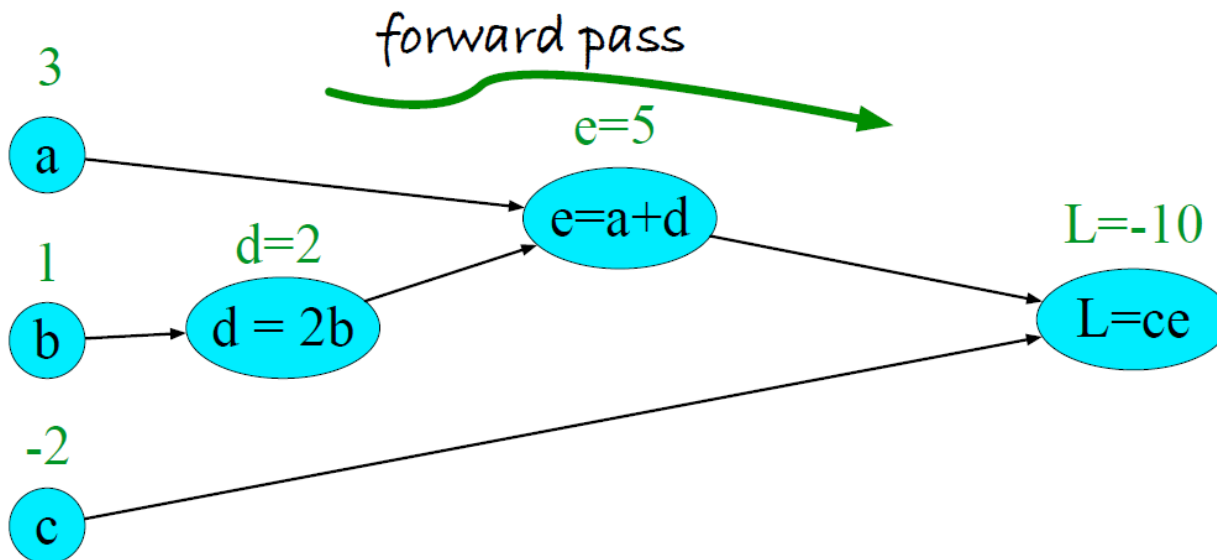
- Regla de la cadena en calculo. Suponga una función compuesta $f(x) = u(v(x))$, la derivada de $f(x)$ por regla de la cadena es:

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx}$$

- Extendida para mas de dos funciones:

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dw} \cdot \frac{dw}{dx}$$

GRAFOS DE COMPUTACIÓN BACKWARD DIFFERENTIATION



$$\frac{\partial L}{\partial c} = ?$$

$$\frac{\partial L}{\partial a} = ?$$

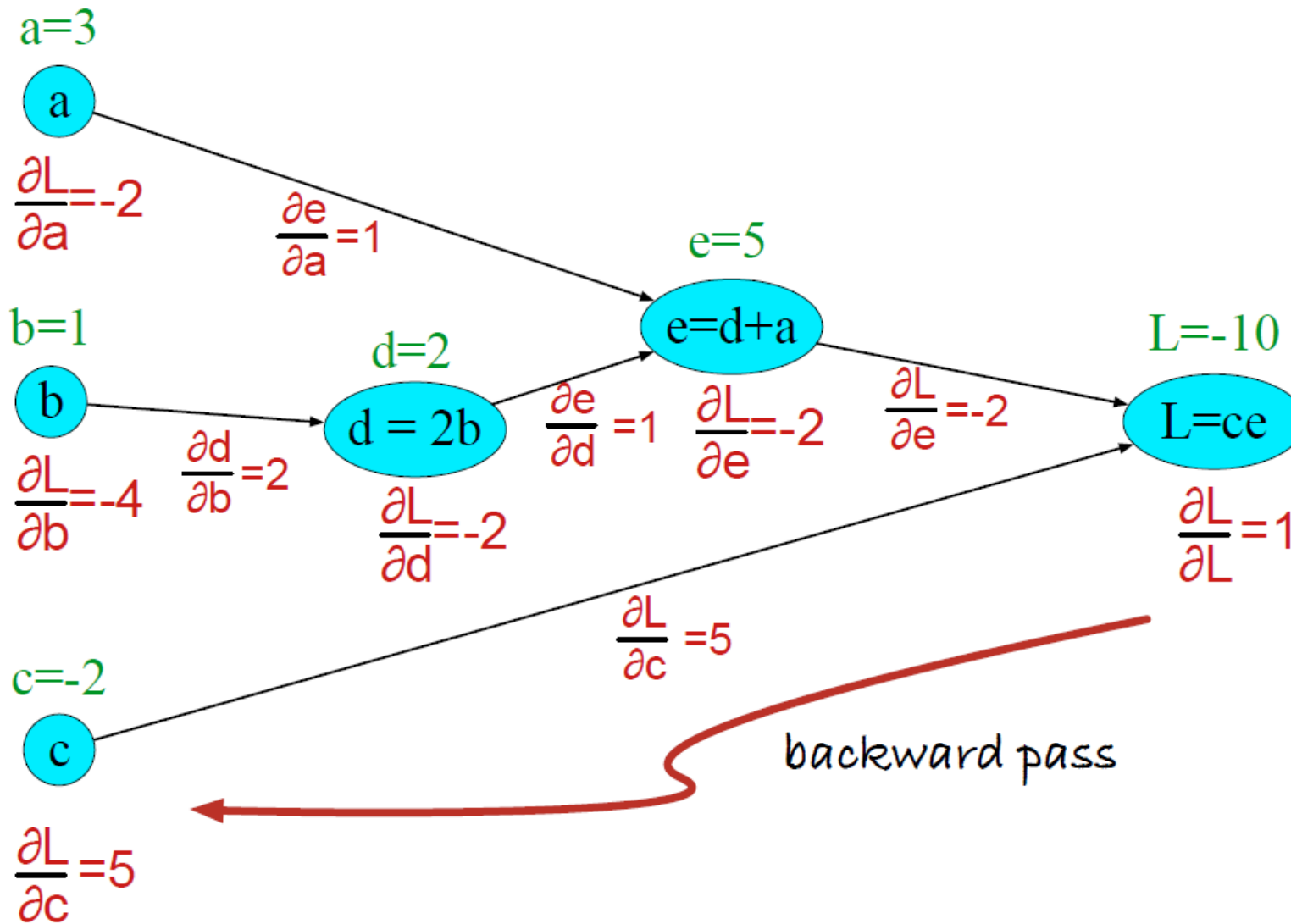
$$\frac{\partial L}{\partial b} = ?$$

$$\frac{\partial L}{\partial c} = e = 5$$

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial a} = (c) * (1) = -2$$

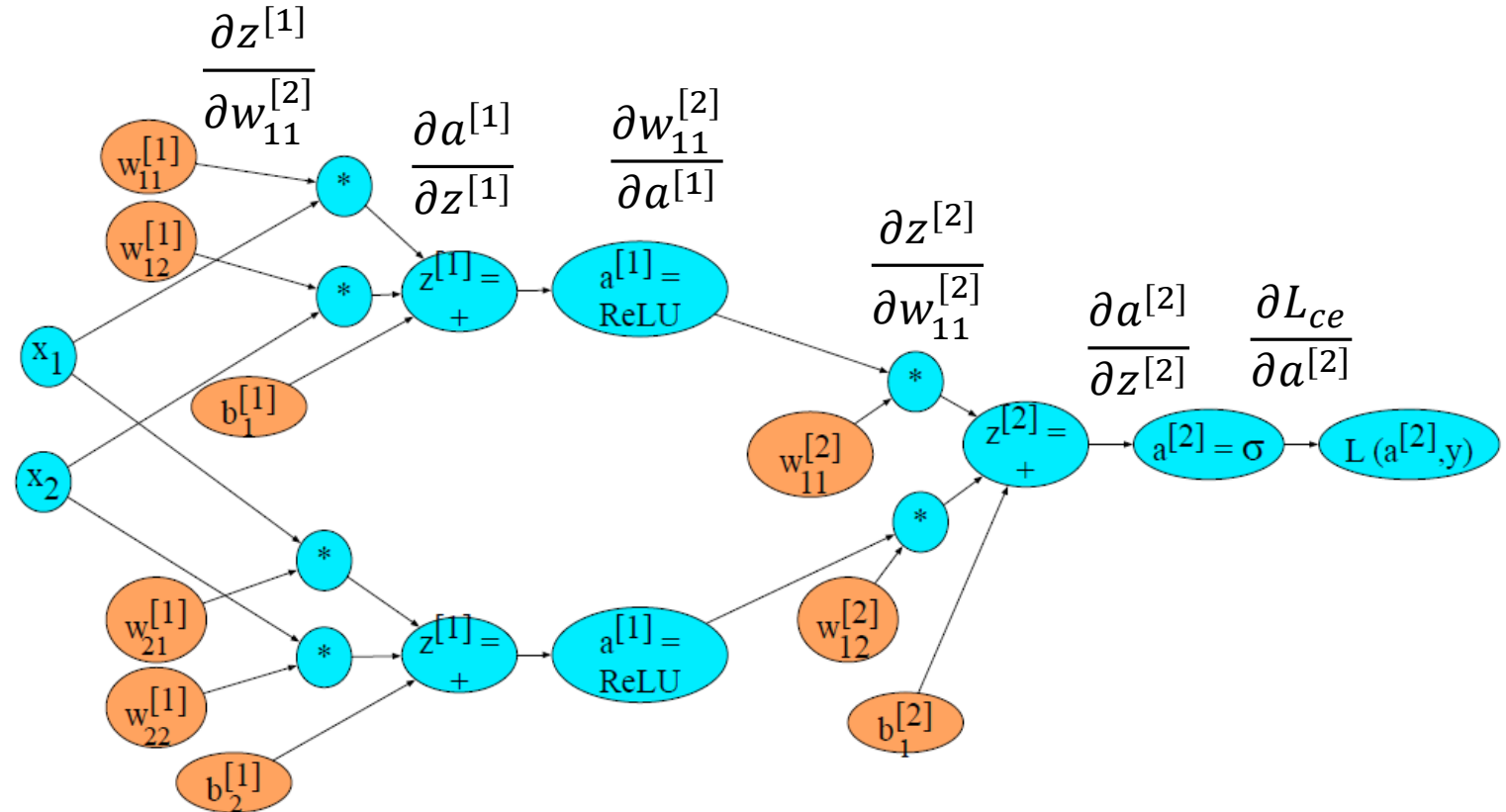
$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial b} = (c) * (1) * (2) = 4$$

GRAFOS DE COMPUTACIÓN BACKWARD DIFFERENTIATION



GRAFOS DE COMPUTACIÓN BACKWARD DIFFERENTIATION

- $\mathbf{z}^{[1]} = \mathbf{W}^{[1]} \mathbf{a}^{[0]} + \mathbf{b}^{[1]}$
- $\mathbf{a}^{[1]} = g^{[1]}(\mathbf{z}^{[1]})$
- $\mathbf{z}^{[2]} = \mathbf{W}^{[2]} \mathbf{a}^{[1]} + \mathbf{b}^{[2]}$
- $\mathbf{a}^{[2]} = g^{[2]}(\mathbf{z}^{[2]})$
- $\hat{y} = \mathbf{a}^{[2]}$



Los gradientes son usualmente
 números menores que cero.
 El gradiente se desvanece en la
 propagación hacia atrás.

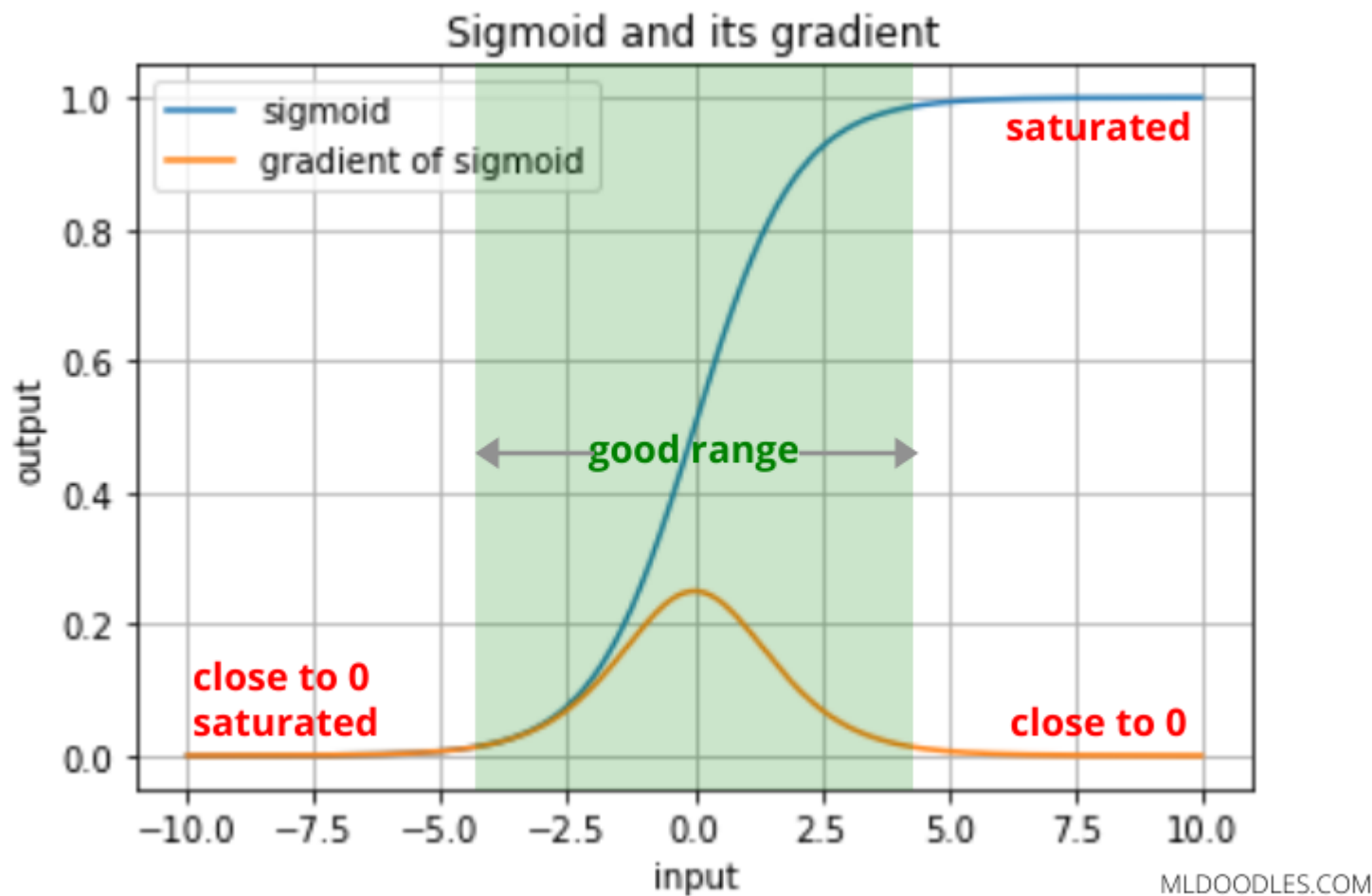
$$\frac{\partial L_{ce}}{\partial w_{11}^{[2]}} = \frac{\partial z^{[1]}}{\partial w_{11}^{[2]}} \frac{\partial a^{[1]}}{\partial z^{[1]}} \frac{\partial w_{11}^{[2]}}{\partial a^{[1]}} \frac{\partial z^{[2]}}{\partial w_{11}^{[2]}} \frac{\partial a^{[2]}}{\partial z^{[2]}} \frac{\partial L_{ce}}{\partial a^{[2]}}$$

Que derivadas necesitamos?

- $\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$
- $\frac{d\tanh(z)}{dz} = (1 - \tanh^2(z))$
- $\frac{dReLU(z)}{dz} = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
- Después del forward aplicamos la diferenciación hacia atrás en el grafo de computo.

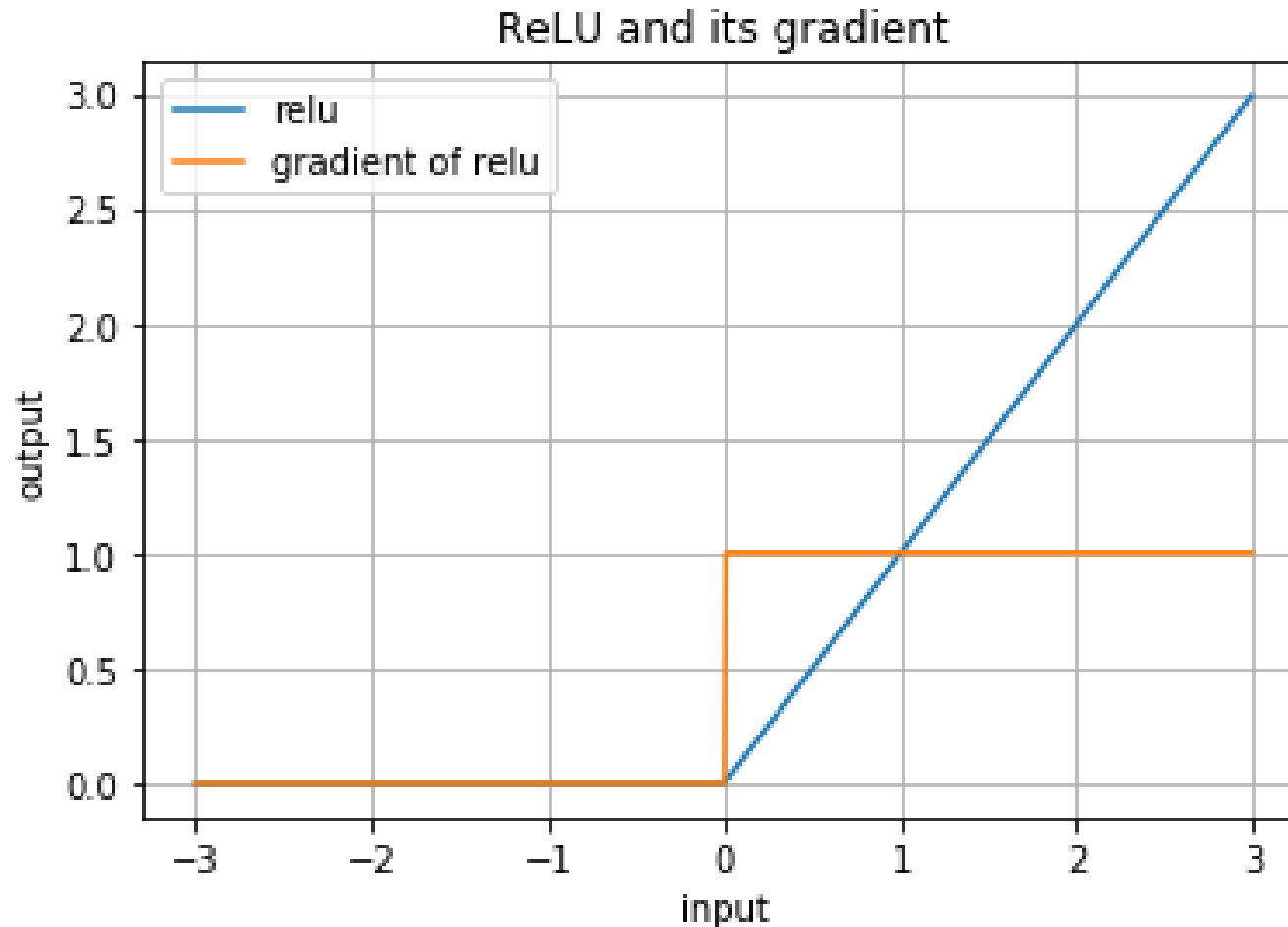
GRAFOS DE COMPUTACIÓN

PROBLEMA DE LA SATURACIÓN DEL GRADIENTE



GRAFOS DE COMPUTACIÓN

PROBLEMA DE LA SATURACIÓN DEL GRADIENTE



ReLU (unidad lineal rectificada) no tiene problemas de saturación.

Se prefiere esta función de activación para las capas intermedias.

Introducción a Tensorflow y Pytorch!